Bijou

The Bijou Book

Build, understand, test and maintain terminal applications

James Ross

One application, four environments

20 October 2026

Copyright

Copyright © 2026 James Ross.

Except for code examples and material identified as excluded or separately licensed, the text, diagrams, and illustrations in this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

This license permits copying, redistribution, translation, and adaptation, including commercial reuse, subject to its attribution and other requirements. Reuse must not imply the author’s endorsement.

License terms:
https://creativecommons.org/licenses/by/4.0/

Unless otherwise indicated, the source code examples in this book are licensed under the Apache License, Version 2.0. These examples are excluded from the Creative Commons license stated above.

License terms:
https://www.apache.org/licenses/LICENSE-2.0

Third-party material remains subject to the rights and terms identified in its accompanying credits or notices. This page does not replace or modify those terms.

The warranty disclaimers and limitations of liability in the applicable licenses apply to the material licensed under them.

The full Apache 2.0 license and reproduction notices appear in Appendix P2. Font software is separately licensed; credits and license files accompany the sources.

Version and reproducibility

The code this edition teaches

This edition uses published Bijou 7.2.0 and the companion's book-v2 tag. Clone the exact edition, run npm ci, then follow the five independent build stages. The original book-v1 remains available. Historical source records describe their original versions and do not change the teaching dependency manifest.

Artifact	Pinned value
Companion	flyingrobots/bijou-job-monitor / book-v2
Companion commit	1c5c47ce0eeeb7342fb7e95e6424a2256c1288a1
Teaching API	Published Bijou 7.2.0
Node / npm	26.10.0 / 11.19.1
Locally tested host	macOS arm64; model, output and pseudo-terminal checks
Book component artwork	Bijou checkout 836878a8, 8.0.0-rc.1
Network renderer demos	bijou-fun 0685ddc993491d309468b4f3ca7bc5e6a6e5f4b0, installed Bijou 7.1.0

Rows 1–7. Actual Bijou table component.

The tested boundary

The companion typechecks and passes 44 tests across eight files. A clean candidate install also ran every stage with the finished source hidden. Four actual source diffs were verified. Six local pseudo-terminal cases exercised keyboard input, resizing, quit and SIGTERM for stages 03, 04 and 05 and verified restoration of canonical-input and echo flags. Those cases observe output and lifecycle; deterministic App and model tests establish input semantics. The recorded host was macOS arm64. Hosted CI status belongs to its recorded run at the pinned commit.

Human observations remain separate

The unfamiliar-reader walkthrough is prepared and has not been conducted. A participant session is planned; no observations have been recorded. Its protocol and blank form are included in the companion and summarized in the book appendix. No screen-reader usability study or broad emulator matrix is claimed. The accessible CLI prints a fixture and exits; an interactive screen-reader workflow is not implemented.

Build stages and inspections

Use npm run stage – 01 through 05, npm run stage:check – 01, and npm run stage:diff – 01 02. Each stage owns its source modules and shares only the root dependency lock. The older checkpoint commands inspect the completed implementation. The workshops distinguish those two routes explicitly.

Report corrections at https://github.com/flyingrobots/bijou-job-monitor/issues, including the edition commit, stage or workshop, command and environment. The fonts used for printed specimens are publication choices, not assumptions about the reader’s terminal.

Preface

A useful terminal application has to keep its promises when its presentation changes. A failed job must still be recognizable without red. A cancellation dialog has to own its keys. A late response must not replace the detail for a different selection. A resize must leave a person a way to finish the task. These are everyday application problems, and they are the questions this book uses to teach Bijou.

We will work through a job monitor. It starts small: three jobs, stable identities and explicit states. We then encounter four environments, keyboard routing, asynchronous work, constrained geometry and the evidence needed to trust each behavior. The application simulates its jobs. That makes races and failures reproducible without requiring the reader to build a process supervisor before learning the interface architecture.

The route through the book

Learn by building

Part I, Why Bijou, explains the principles and their costs. Part II, Build the job monitor, gets the model and its presentations working before following a keypress into a visible frame. Part III, Understand the machinery, explains ownership, effects, late results, layout and semantic styling.

Part IV, Make it dependable, puts the application under pressure, matches tests to claims and provides a troubleshooting route. Part V, Ship and maintain it, establishes the reproducible edition and walks through a maintenance change. The component atlas, renderer gallery, glossary and topic finder form a reference section after the teaching text.

Choose a route that fits your work

If this is your first Bijou application, read the five Parts in order and run each independent build stage. If you already have an application with an input or asynchronous defect, begin with the keypress trace, the late-result trace and the troubleshooting chapter. Component authors should read the layout and styling chapter, then consult the atlas with its version boundary in mind. Framework contributors can use the same teaching route to identify the application contract before following the corresponding archive record.

A working example, an explicit version

Read and run together

The companion repository is https://github.com/flyingrobots/bijou-job-monitor. The version page fixes this edition to a published Bijou release and an immutable companion commit. Five independent build stages let readers construct the application through actual source changes. The older checkpoint commands remain reproducible observations of the completed modules. The broken exercises have observable failures and repaired paths. Their purpose is to make a design decision testable before its explanation becomes comfortable vocabulary.

The book's figures and source listings have different jobs. A component specimen shows actual rendered cells. A TikZ trace explains causality or ownership. A listing names the code involved. None of those artifacts, by itself, establishes every behavior of the running application. We will say what each check actually observes.

The archive has its own place

History supports the lesson

Earlier editions assembled the project's working documents into a large collection. This edition gives the teaching text its own structure. All 426 source documents remain in the companion Design Archive and in the complete edition's historical appendices. They include guides and principles as well as proposals, releases, retrospectives and audits. Every archived record is marked as historical; its original status has not been independently reclassified as a current implementation claim.

Use the archive to examine an argument or follow a decision into its recorded evidence. Use the teaching book and pinned companion to learn the application in this edition. Exact original Markdown remains in the frozen source corpus, while the typeset records preserve the prior edition's illustrated presentation. A published proposal is useful history. Whether its promise holds in a particular release remains a question for that release's code and evidence.

The separate *Bijou Design Archive* PDF contains those historical appendices. The complete edition combines both volumes without maintaining another copy of their chapter sources.

About this edition

The publication has three outputs from one modular LaTeX source tree: *The Bijou Book*, *Bijou Design Archive*, and *Bijou Complete Edition*. The main book contains five teaching Parts, seven runnable workshops and seven connecting chapters. Historical source records appear only in the archive and the complete edition's historical appendices. The main book includes a component atlas, renderer gallery, glossary, topic finder, exercise answer key and a prepared unfamiliar-reader walkthrough. Companion book-v2 adds five independently runnable stages and actual adjacent source diffs. No human walkthrough results are claimed.

All 426 source records and the frozen corpus survive this restructuring. Their prior typeset content, native TikZ figures and component-rendered tables remain unchanged. The 32 Mermaid conversions and 83 arrow-notation figures belong to those historical records; the two interaction traces belong to the workshops. Tables continue to use actual Bijou component output. Artwork comes from three explicitly distinct contexts: the 7.2.0 companion, the 8.0.0-rc.1 component capture checkout and the separate 7.1.0 renderer demonstrations.

Jacquard 24 supplies titles and all heading levels. IBM Plex Mono supplies prose, VT323 source listings, and Recursive Mono terminal specimens and component tables, with DejaVu Sans symbols. The existing coral palette is retained. Original font licenses, source hashes, capture parameters and code-example notices accompany the sources.

The version page records what was tested for the companion. New connecting chapters explain the same implementation and propose clearly labeled reader exercises. They do not claim that the example implements a process runner, a complete localization catalog, a built-in inspector or a verified human screen-reader workflow.

Contents

Version and reproducibility ii

Preface iv

About this edition vi

I Why Bijou 1

Introduction: A useful promise 2

1 Preserve the user’s meaning 3

2 Choose Bijou deliberately 6

II Build the job monitor 9

Introduction: Start with a task you can inspect 10

3 Build from a working stage 11

4 Workshop 1: A job worth monitoring 14

5 Workshop 2: One application, four environments 17

6 Workshop 3: Follow one keypress 20

III Understand the machinery 23

Introduction: Give every change an owner 24

7 Find the application boundaries 25

8 Workshop 4: Results arrive after the screen changes 30

9 Turn meaning into cells 33

IV Make it dependable 35

Introduction: Make the promises observable 36

10 Workshop 5: When the terminal fights back 37

11 Workshop 6: How do we know it works? 39

12 Troubleshoot from the symptom 42

V Ship and maintain it 45

Introduction: Keep the example reproducible 46

13 Workshop 7: Ship a reproducible example 47

14 Maintain the promise after release 49

Reference 52

A The component cell atlas 54

B Renderer experiments from bijou-fun 77

C Glossary 80

D Find a practical answer 84

E Exercise answer key 86

F An unfamiliar reader walkthrough 90

P1 Sources and further reading 92

P2 Licenses and reproduction notices 93

Why Bijou

A useful promise

Before choosing components, decide what the application owes its user. These chapters establish the promises that the job monitor must preserve and the cost of using an application framework to keep them.

What you will be able to do

At the end of this Part, write one sentence stating the task your application serves, name the environments in which it must work, and explain why a stateful terminal interface earns its complexity.

1

Preserve the user's meaning

Start with an answer someone needs

What the operator came to learn

Imagine that a build has stopped producing output. The operator needs to distinguish a slow job, a failed job and a disconnected presentation. A spinning marker may make the application look active while answering none of those questions. Start the design with facts the person needs: the job identity, its state, the last known progress, any failure explanation, and the actions that remain available.

Our example has compile, test and publish jobs. Their names describe a familiar workflow; their execution is simulated. This keeps the learning problem small while preserving the architectural pressures: several things change over time, results arrive out of order, a selected item has detail, and cancellation needs an owner.

An observation before a component

Write an acceptance observation before choosing a widget: “With color removed, I can identify the failed job and read its explanation.” A badge might help fulfill that promise. It cannot define the promise by itself. Another observation is “Opening confirmation prevents movement behind it.” That property belongs to input routing, even if the dialog drawing is perfect.

These observations are deliberately concrete. They can be challenged with a saved stream, a controlled transition or a keyboard session. “The interface is polished” does not tell the next engineer which behavior to preserve during a repair.

One meaning can have several presentations

Four environments, four contracts

Interactive mode offers a loop, selection and actions. Static mode offers a completed snapshot suitable for a report or a CI log. Pipe mode offers stable

fields that another program can consume. Accessible mode states important relationships explicitly in readable prose. In this companion, the noninteractive modes render the fixed example snapshot and exit; they do not observe live background jobs.

The shared model is valuable because each presentation starts with the same facts. It does not make the presentations correct automatically. A pipe formatter can still omit the error field. A textual presentation can still forget to name the selected job. The useful test compares the facts each mode promises to preserve.

Honest degradation is a design decision

At a narrow width, our application gives up a simultaneous multi-column comparison and uses compact records with scrolling. This is a chosen loss. The job's identity, state and error remain available. If the task required comparing a hundred jobs at once, that choice might be inadequate even though every painted cell fitted its rectangle.

The right fallback depends on the task. Before compressing a view, name what it may lose and what it must retain. There may be a minimum useful size below which the application should explain how to recover. A renderer's smallest legal Surface is not a promise that the workflow is useful at that size.

Make the boundaries visible

Facts, intentions and effects

A job is running because the model says it is running. A keypress becomes an intention such as start or dismiss. The model returns a new state and descriptions of work to perform. An adapter owns timers and emits later results. A presentation reads the state and produces output. Each boundary makes a different question inspectable.

This separation costs code and vocabulary. A tiny report may not need it. The cost begins to earn its place when a keyboard event, a background result and a resize can arrive independently. Explicit boundaries let us reproduce those situations without depending on luck or wall-clock timing.

The framework supplies tools; the application supplies policy

Bijou supplies the runtime and rendering contracts used by the example. The companion decides which keys are global, what cancellation means, how modes are chosen and which result still belongs to a screen. Treat those policies as application design. A framework cannot infer whether leaving a log pane should cancel the underlying build.

A small design exercise

Write the contract in plain language

Choose another task: a download queue, deployment review or backup monitor. Identify three facts that must survive redirected output, one action that needs confirmation, and one result that could arrive after its owner changes. Describe observable success without naming a component. Keep that description beside the implementation; it gives future visual and architectural changes something concrete to answer to.

2

Choose Bijou deliberately

When a terminal application earns its complexity

A changing task with useful interaction

The job monitor benefits from a persistent view because an operator can start work, inspect detail and cancel a selected run while other information changes. A terminal application can keep that work close to a command-line workflow and remain usable over a remote session. Bijou's state, effect and Surface contracts give us a structured way to build it.

The structure is useful when the application has enough concurrent concerns to need explicit ownership. A dialog changes input policy. A log request has a lifetime. A resize changes geometry. A pipe invocation needs another output contract. Those concerns exist whether a framework names them or application code handles them informally.

When plain output is sufficient

A command that computes one answer and exits may be served well by a few lines of ordinary output. A stable machine-readable report may need a serializer and useful exit statuses more than an interactive loop. Start there if it fulfills the task. Choosing Bijou should buy a useful application contract, not merely a decorative border.

Our example deliberately supports simple outputs alongside interaction. Their presence is part of serving different consumers. It also creates maintenance obligations: fields, textual states and error explanations must keep their meaning as the rich presentation evolves.

Understand what you are taking on

Terminal cells and input conventions

The application works in cells whose width is constrained by terminal conventions. Wide and combining characters complicate measurement. Shortcuts need a focus policy. Raw input and terminal restoration have lifecycle

consequences. Remote sessions and CI streams may provide a different environment from a local interactive terminal.

Bijou makes some of those mechanics reusable. The application still chooses its useful minimum geometry, communicates its actions and verifies the environments it claims to support. Tests against controlled cells cannot establish every emulator’s treatment of every Unicode sequence.

Dependencies and version boundaries

The book teaches published Bijou 7.2.0 through an exact companion commit. Its component atlas was captured from an 8.0.0-rc.1 checkout. The renderer demonstrations use 7.1.0. These artifacts answer different questions: how the teaching application behaves, what the captured component surfaces look like, and what particular renderer experiments produced.

Do not copy an unfamiliar API from a historical design or an atlas caption and assume it belongs to 7.2.0. Check the installed package’s exports and types. A proposed architecture can be useful reasoning while naming a contract that changed before publication.

Keep the first application bounded

Why the jobs are simulated

Launching and controlling real processes introduces its own domain: output buffering, signals, process trees, partial results, credentials and exit status. The companion substitutes scheduled work so that progress, failure and cancellation are deterministic teaching material. It never invokes a compiler or publishes a package.

A real process adapter is a reasonable later project. It deserves an explicit contract and its own acceptance evidence. The ability to cancel a scheduled callback does not demonstrate that a child process and all its descendants were stopped.

Define an exit condition for the learning exercise

By the end of the book, you should be able to reproduce the edition, trace input and results, explain each presentation, diagnose the broken exercises and identify the boundaries of the tests. That is a complete learning outcome. A process supervisor, full translation catalog and integrated state inspector remain possible extensions with separate scope.

Before you start

Use the version page to obtain the companion commit. Read its README and verification record. Run npm ci and npm run verify. If reproduction fails, record the environment and exact command before changing the dependencies.

Preserving the failing reference is more useful than silently replacing the edition with whatever a package manager resolves today.

II

Build the job monitor

Start with a task you can inspect

The first three workshops turn a small job model into four presentations and then an interactive application. Follow the same facts through each view. The examples use the published release recorded near the front of the book.

What you will be able to do

Run build stages 01, 02 and 03 and their acceptance checks. Read the adjacent source diffs as you move between them. You should be able to identify a job independently of its label, explain what each mode preserves, and follow Enter from a key message to a running job.

3

Build from a working stage

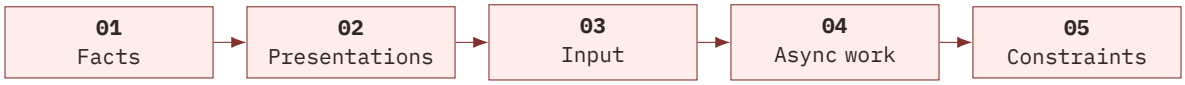
Start with an edition you can return to

Install once, change the application in steps

The companion now contains five independent source snapshots. Each stage is a complete implementation of its declared behavior. It imports neither the finished src/ application nor another stage. All stages share the root package lock and pinned toolchain, so installing five copies of the dependencies is unnecessary.

```
git clone --branch book-v2 --depth 1 https://github.com/flyingrobots/bijou-job-monitor.git
cd bijou-job-monitor
git switch -c my-monitor
npm ci
npm run stage -- 01
npm run stage:check -- 01
```

The version page gives the immutable commit behind the tag. A personal branch gives experiments a home without moving the published edition. The staged source lives under stages/; the final application remains under src/. The original checkpoint commands are still useful views of that final application. They are distinct from the progressive implementations.



Read a real change before applying it

Use adjacent source diffs

Run npm run stage:diff – 01 02. The result is an actual unified diff of source modules. Added and removed lines show how a working earlier stage becomes the next one. The four patches cover 37, 123, 291 and 89 lines respectively, including context. The asynchronous step spans several small modules; its line count is not a promise that every step is equally easy.

Read that larger step by responsibility: protocol, monitor, effect driver, App and host entrypoint, then presentation and fixtures. Each stage's tests

live in its own test/ directory. The source patches deliberately exclude tests and guide changes; inspect those files alongside the implementation.

Keep the old stage runnable

Use a learning copy to type a change or compare it with the next snapshot. Run the previous stage's checks before leaving it, then the next stage's checks. If an experiment stops working, return to its unchanged snapshot rather than silently importing a later module to make an earlier program pass. Independent snapshots are deliberate duplication here: they preserve a useful teaching boundary at a fixed edition.

Know exactly what each stage adds

Stages 01 and 02: facts and presentations

Stage 01 prints three queued jobs with stable identities and zero progress. It validates impossible progress and duplicate IDs. Stage 02 adds a shared completed/running/failed fixture and four formatters. All four outputs print once and exit. Its interactive output is a content specimen; keyboard input arrives in stage 03. Completion means the status, progress and failure explanation survive every promised presentation, with four fields in each TSV row.

Stage 03: synchronous interaction

Run npm run stage – 03 in a real terminal. Up and Down select; Enter marks a job running; C opens confirmation; Enter confirms; Escape leaves it running. Q and Ctrl+C quit globally. There is no scheduler, log loader or filter yet. A running job stays running until cancelled or the application closes. The stage's update returns a new Monitor directly.

The simple view supports 80 by 24 cells. Below that it displays a resize instruction while retaining global quit. That is an explicit supported boundary, not the completed compact-layout implementation. The input tests establish modal ownership independently of the picture.

Stage 04: work has identity and a lifetime

Stage 04 adds simulated progress, failure, log requests and filtering. Its transition now returns a model and effect descriptions. The driver owns scheduled work; run IDs and view epochs reject stale results. Its simple view targets 80 by 32 cells and provides a resize recovery prompt below that boundary. Start with npm run stage – 04, then npm run stage:check – 04.

A deterministic scheduler makes completion, cancellation and disposal observable without sleeping. The previous synchronous program remains valid on its own terms; adding asynchronous work introduces new obligations rather than retroactively proving it had them.

Stage 05: the task survives a smaller presentation

Stage 05 adds compact records, grapheme-aware wrapping, scrolling and selected-item visibility. Resize it to 24 by 8 cells and inspect what remains reachable. Its eight application modules match the final src/ implementation byte for byte. One-cell tests establish bounds; they do not establish that a human can complete the whole task at one cell.

Use hints without losing the diagnosis

Attempt the broken exercises before opening the answer key. Each answer gives a hint, diagnosis, repair, a tempting failed alternative and an observable completion criterion. The walkthrough appendix supplies prompts for an unfamiliar reader. Automated reproduction of a clean installation is evidence about installation; it is not a substitute for that person’s comprehension.

4

Workshop 1: A job worth monitoring

Start with the user's question

A person starts a build and wants to know whether it is making progress, whether it failed, and whether it is safe to leave. The job monitor in these workshops gives those questions a concrete home. It has three jobs: compile the application, run the tests, and publish the package. These are simulated jobs driven by scheduled events. The example never invokes a build tool or publishes anything. That boundary keeps the lesson about terminal application design while leaving room for a real process adapter later.

Facts before decoration

A job has a stable identity, a title, a textual status, a percentage and a run identity. Failure can also carry an explanation. The run identity distinguishes two attempts at the same job; a percentage alone cannot do that. Selection belongs to the monitor, rather than being inferred from where a highlight happened to be painted.

The Job class checks its invariants when a value is created. A successful job must be complete. Progress must be a finite number between zero and one hundred. A run identity is a non-negative integer. An immutable job can be handed to a renderer or retained in a test without a later update silently changing the value under examination.

```
// Copyright 2026 James Ross. SPDX-License-Identifier: Apache-2.0
import type { JobStatus } from './protocol.js';
export class Job {
  public constructor(
    public readonly id: string,
    public readonly title: string,
    public readonly status: JobStatus = 'queued',
    public readonly percent = 0,
    public readonly run = 0,
    public readonly error: string | undefined = undefined,
  ) {
    if (!id.trim() || !title.trim()) throw new Error('Job identity and title are required');
```

```
    if (!Number.isFinite(percent) || percent < 0 || percent > 100) throw new Error('Progress
    ↪  must be 0..100');
    if (!Number.isSafeInteger(run) || run < 0) throw new Error('Run identity must be a
    ↪  non-negative integer');
    if (status === 'succeeded' && percent !== 100) throw new Error('Successful jobs must be
    ↪  complete');
    Object.freeze(this);
  }
  public start(): Job { return new Job(this.id, this.title, 'running', 0, this.run + 1); }
  public cancel(): Job { return new Job(this.id, this.title, 'cancelled', this.percent,
  ↪  this.run + 1); }
}
```

What immutability costs

Each transition constructs a new job or monitor snapshot. That adds allocation and some copying. For three small jobs, the cost is easy to understand and measure. For a monitor with hundreds of thousands of entries, the collection strategy would need another decision. Immutability establishes an ownership boundary; it does not exempt a program from measuring its workload.

Keep meaning out of the presentation

A failed job remains failed when the terminal loses color. A running job remains running when a narrow viewport requires a different layout. The domain model therefore stores statuses such as running and failed, not color names, terminal escape sequences or coordinates.

A tempting shortcut

It is tempting to store preformatted rows as the application’s state. That works until filtering needs the job identity, cancellation needs its run, or a pipe consumer needs a numeric percentage. Parsing those facts back out of painted text reverses the dependency. Here, presentation consumes the facts. Commands and tests use those same facts directly.

Run the first build stage

From the companion repository, run npm ci, then npm run stage – 01. The output contains three queued jobs. Read the rows with color disabled and answer: what stable identity does each job have, what state is it in, and how much work is complete? Stage 01 introduces its own stages/01-model/job.ts, monitor snapshot and plain presentation. Run npm run stage:check – 01 before reading the 01-to-02 source diff. The next workshop reuses that snapshot rather than inventing a separate model for each environment.

Try a real extension

Add a job whose title contains a wide character and a combining accent. Give it an identity that remains stable if its title changes. Success means the same identity reaches every presentation and every transition. A title is a label for people; changing its spelling must not redirect an asynchronous result to another job.

5

Workshop 2: One application, four environments

Hold the task constant

Run npm run stage – 02 and its stage check. Every presentation receives the same fixture: compilation succeeded, tests failed with an explanation, and publication is forty percent complete. Changing a presentation must not change those facts. The following specimens show the completed companion's actual formatter and Bijou 7.2.0, using Recursive Mono for the terminal cell projection. Stage 02 supplies a smaller independent formatter with the same facts; the figures are a reference target, not a claim that its output is byte-identical. They are complete fixture outputs wrapped at forty cells for print. The interactive specimen shows presentation content; input routing and the runtime viewport are introduced in Workshop 3.

Interactive and static

Interactive

```
Job monitor

Filter: all jobs

  compile: succeeded 100%
  test: failed 60%
  Fixture failure: assertion did not
pass
> publish: running 40%

Up/Down: select · Enter: start · c:
cancel · l: logs · f: failures · q: quit

test: Fixture failure: assertion did not
pass
```

Static / CI

```
Job monitor

Filter: all jobs

  compile: succeeded 100%
  test: failed 60%
  Fixture failure: assertion did not
pass
> publish: running 40%

Up/Down: select · Enter: start · c:
cancel · l: logs · f: failures · q: quit

test: Fixture failure: assertion did not
pass
```

The rich mode provides an application loop in a terminal. Static mode emits a snapshot suitable for a CI log or a saved report. The two narrow specimens deliberately share a compact textual layout. The important difference is the delivery contract: a static invocation writes once and exits; an interactive invocation accepts input and redraws a viewport.

Pipe and accessible

Pipe

```
id →⇥ status →⇥ progress →⇥ error
compile →⇥ succeeded →⇥ 100 →⇥
test →⇥ failed →⇥ 60 →⇥ Fixture failure:
assertion did not pass
publish →⇥ running →⇥ 40 →⇥
```

Accessible

```
Job monitor
Compile the application. Status:
succeeded. Progress: 100 percent.
Run the tests. Status: failed. Progress:
60 percent. Error: Fixture failure:
assertion did not pass
Selected. Publish the package. Status:
running. Progress: 40 percent.
Up and Down select. Enter starts. C
opens cancellation. L opens logs. F
filters failures. Q quits.
```

The pipe specimen makes tab characters visible as →⇥ for print. Its original TSV output is retained with the capture. It has stable identifiers, textual statuses, numeric progress and a separate error field. It has no cursor controls, decorative frame or keyboard help. The accessible presentation spells out selection, state, progress and the available actions. Its CLI mode prints the fixture and exits; the action descriptions do not establish an accessible interactive session. That is an explicit alternative presentation, not a claim that an automated text check establishes usability with every screen reader.

Degrade honestly

Preserve the answer, choose the form

A narrow terminal may replace a multi-column table with one compact record per job. A pipe should replace visual relationships with a stable serialization. An accessible view should state a selected relationship that a highlighted row might otherwise imply. In each case, ask what the reader still needs to understand and accomplish. The invariant is the task’s useful meaning.

Choose mode and styling separately

The companion uses an explicit –mode choice when provided. Otherwise, redirected output chooses pipe mode; a CI environment attached to a TTY chooses static mode; an ordinary TTY chooses interactive mode. NO_COLOR controls styling. It is not an instruction to erase interaction or replace a record format. These are application decisions made around the pinned Bijou API, not an assumption that every historical release detected modes in the same way.

For exact TSV without npm’s script banner, use node –import tsx src/main.ts –mode=pipe > jobs.tsv, or npm’s silent form documented in the companion. Inspect the saved file with a tab-aware tool. A terminal screenshot cannot establish that a data stream has the right number of fields.

Break meaning before repairing style

Run npm run exercise – color. It fails because a colored dot is the only failure indication. Remove the color and the assertion has nothing useful to read. Diagnose the missing fact, then run with –repair. The repair uses a textual status. Color can reinforce that word later. Its cost is a less compact display; its benefit is that failure survives monochrome output, copying and speech.

6

Workshop 3: Follow one keypress

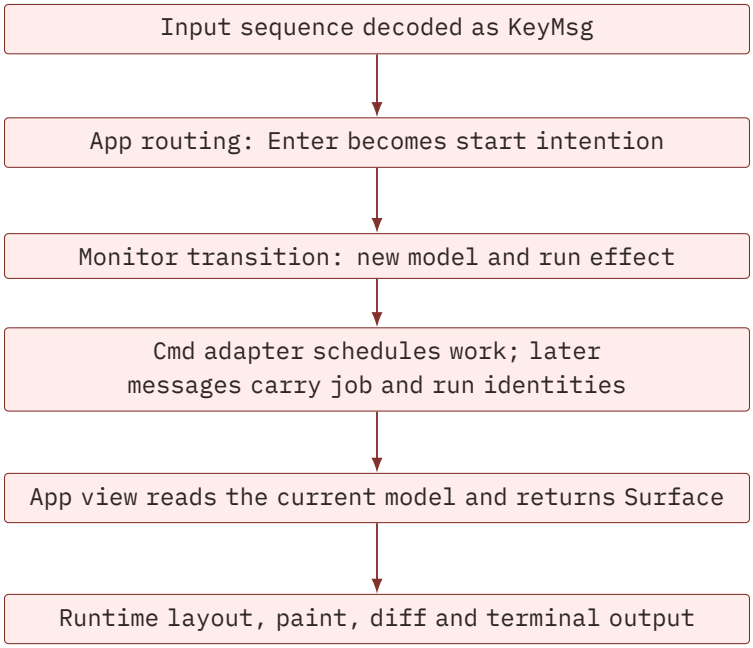
Run the application

Run npm run stage – 03 in a terminal. Up and Down select jobs. Enter marks the selected job running. C opens cancellation; Enter confirms and Escape leaves it running. Q or Ctrl+C quits globally. There are no timers, logs or filtering yet, and the supported view is 80 by 24 cells. Run the stage's input checks before moving on.

The complete trace below follows the final application. Its asynchronous effects arrive in stage 04; log, filter and scroll actions are available in the later implementation. Stage 03 returns a model directly, while the later transition also describes work to perform. Keeping that difference explicit lets the small program teach input ownership before scheduling is introduced.

Name the intermediate representations

A terminal input sequence is decoded by Bijou into a KeyMsg. The application's routing function translates it into a monitor intention. The immutable monitor transition produces a new model and explicit effects. An effect adapter schedules work and emits later messages. The view reads the current model and returns a Surface. Bijou's runtime owns layout handling, painting, diffing and terminal output. These are the App, Cmd and Surface contracts verified against the published 7.2.0 release.



Trace Enter on the compile job

The input message identifies Enter. The application turns it into {type: 'start'}. The monitor constructs a running job with a new run identity and zero progress. Its effect identifies the job and that exact run. The current model can now render "running, 0 percent" before the simulated work completes. Later progress messages contain the same job identity and run. A completion updates that run's state; it cannot safely identify its target by row number after filtering or selection changes.

The companion's src/app.ts is the adapter between framework messages and monitor intentions. src/monitor.ts owns business transitions. src/effects.ts owns scheduled work. That division lets a model test ask what changed without starting a terminal, while a routing test can still verify that a real framework key reaches the intended transition.

Give input an owner

A modal is an input boundary

The confirmation dialog blocks selection, filtering, starting work and opening logs behind it. Progress messages can still update jobs. Escape dismisses the dialog; it does not send a cancellation effect. Confirming cancellation changes the job state, invalidates its old run identity and asks the adapter to stop scheduled work. If the job finished while the dialog was open, confirmation cannot turn that successful completion into a cancelled job.

A tempting implementation handles every shortcut globally, then draws a modal on top. It looks correct in a screenshot while the keys still affect the page

underneath. Explicit routing adds a branch and a policy that must be maintained. The cost is real, but so is the failure it prevents.

Reproduce the leak

Run npm run exercise – modal. The faulty route clears the dialog boundary before applying movement, so the selected job behind it changes. The success criterion is a state observation: selection remains unchanged while the modal owns movement. Run the repair and inspect the corresponding production transition. A painted overlay alone cannot establish that guarantee.

Know what a frame proves

The returned Surface gives a test controlled cells and dimensions to inspect. It does not prove that Escape is routed correctly or that a cancelled operation stopped. Those claims need the transition and lifecycle evidence introduced in the next workshops. Keep those claims separate even when the same feature has a convincing screenshot.

III

Understand the machinery

Give every change an owner

The working application now has enough behavior to expose its boundaries. We will distinguish the framework from the application, examine a stale response, and connect useful content to terminal cells.

What you will be able to do

You should be able to name the owner of input, a job run, a log request and a rendered frame. Use the two traces to explain why cancellation and stale-result rejection are separate obligations.

7

Find the application boundaries

Read the running system in layers

The model answers what is true

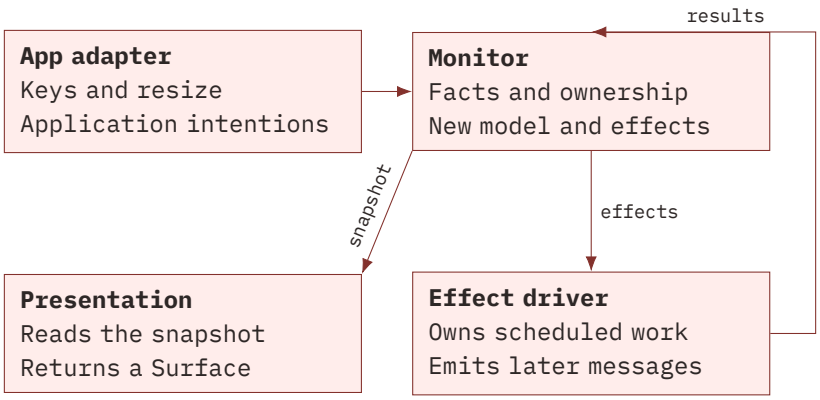
The companion has two useful levels of state. Monitor owns jobs, selection, filters, confirmation, the current detail screen and its request epoch. AppModel adds viewport dimensions and scroll offset. The first level expresses the task; the second connects it to a terminal application.

A domain transition returns a model and effect descriptions. Starting a job records a running state and a new run identity, then requests work for that run. The view can represent the new state before completion arrives. No test has to inspect a spinner to decide whether the transition happened.

The adapter answers what should happen outside the model

The effect driver turns descriptions into scheduled operations. Its scheduler port supplies a cancellation function. A deterministic scheduler in the tests can advance those operations without sleeping; the host adapter uses timers. This is the practical purpose of the port: application decisions can be tested without making host timing part of every assertion.

An effect description is not evidence that work completed. It names an instruction to the adapter. A later message reports an outcome, and the model checks whether that outcome still belongs to the relevant run or view.



Inspect the actual adapter

An App connects framework messages to intentions

The following complete file is reproduced from the pinned companion. init, update and view form the App contract used here. Keyboard messages are translated into monitor events. Resize updates the viewport. Later domain messages return through the same update boundary.

```
// Copyright 2026 James Ross. SPDX-License-Identifier: Apache-2.0
import type { BijouContext } from '@flyingrobots/bijou';
import { isKeyMsg, quit, type App, type Cmd, type KeyMsg } from '@flyingrobots/bijou-tui';
import { EffectDriver } from './effects.js';
import { Monitor } from './monitor.js';
import type { MonitorEvent } from './protocol.js';
import { view } from './presentation.js';
export interface AppModel { readonly monitor: Monitor; readonly width: number; readonly height:
↪  number; readonly offset: number }
export function keyIntent(key: KeyMsg): MonitorEvent | undefined {
  if (key.key === 'up') return { type: 'move', delta: -1 };
  if (key.key === 'down') return { type: 'move', delta: 1 };
  if (key.key === 'enter' || key.key === 'return') return { type: 'start' };
  if (key.key === 'c') return { type: 'ask-cancel' };
  if (key.key === 'escape') return { type: 'dismiss' };
  if (key.key === 'f') return { type: 'filter' };
  if (key.key === 'l') return { type: 'logs' };
  return undefined;
}
export interface ShutdownPort { subscribe(stop: () => void): () => void }
export type AppEvent = MonitorEvent | { readonly type: 'shutdown' };
export function createMonitorApp(ctx: BijouContext, driver: EffectDriver, shutdown?:
↪  ShutdownPort): App<AppModel, AppEvent> {
  return {
    init: () => [{ monitor: Monitor.initial(), width: ctx.runtime.columns, height:
    ↪  ctx.runtime.rows, offset: 0 }, shutdown ? [emit => shutdown.subscribe(() => emit({ type:
    ↪  'shutdown' }))] : []],
    update: (message, model) => {
      if (message.type === 'shutdown') { driver.dispose(); return [model, [quit()]]; }
      if (isKeyMsg(message)) {
        if (message.key === 'q' || (message.ctrl && message.key === 'c')) { driver.dispose();
        ↪  return [model, [quit()]]; }
        if (!model.monitor.confirming && (message.key === 'j' || message.key === 'k')) return [{
        ↪  ...model, offset: message.key === 'j' ? model.offset + 1 : Math.max(0, model.offset -
        ↪  1) }, []];
        const intent = model.monitor.confirming && (message.key === 'enter' || message.key ===
        ↪  'return') ? { type: 'confirm-cancel' as const } : keyIntent(message);
        if (!intent) return [model, []];
```

```
        const transition = model.monitor.update(intent);
        const commands: Cmd<AppEvent>[] = transition.effects.map(effect => emit => {
        ↪  driver.execute(effect, emit); return undefined; });
        return [{ ...model, monitor: transition.model, offset: 0 }, commands];
      }
      if (message.type === 'resize') return [{ ...model, width: Math.max(1, message.columns),
      ↪  height: Math.max(1, message.rows), offset: 0 }, []];
      if (message.type === 'mouse' || message.type === 'pulse') return [model, []];
      const transition = model.monitor.update(message);
      return [{ ...model, monitor: transition.model }, transition.effects.map(effect => emit => {
      ↪  driver.execute(effect, emit); return undefined; })];
    },
    view: model => view(model.monitor, ctx, model.width, model.height, model.offset),
  };
}
```

Read policy where it is implemented

Notice the global quit route and the different Enter intention while confirmation is open. The model also guards the confirmation boundary against movement, filtering and new starts. That policy is reinforced at two meaningful seams: the adapter translates input, and the model protects its transition contract when events originate elsewhere.

Scroll offset is presentation state. A job’s run identity is task state. Mixing them would make a terminal resize capable of changing the identity of ongoing work. Keeping them separate helps a test make the right observation at the right level.

The host chooses presentation and shutdown

Make the delivery contract visible

The complete entrypoint below shows the application policy at the process boundary. It selects a mode, validates an explicit choice, writes one noninteractive snapshot or starts the interactive runtime, and connects process signals to shutdown. The surrounding context supplies host services; the Monitor remains independent of the process environment.

```
// Copyright 2026 James Ross. SPDX-License-Identifier: Apache-2.0
import { createNodeContext } from '@flyingrobots/bijou-node';
import { run } from '@flyingrobots/bijou-tui';
import { createMonitorApp } from './app.js';
import { EffectDriver } from './effects.js';
import { exampleSnapshot } from './fixtures.js';
import { present } from './presentation.js';
const args = process.argv.slice(2);
const modeArgument = args.find(value => value.startsWith('--mode='))?.slice(7);
```

```
if (modeArgument && !['interactive', 'static', 'pipe', 'accessible'].includes(modeArgument))
↪  throw new Error('Use --mode=interactive|static|pipe|accessible');
// Presentation mode is an application choice. NO_COLOR only controls styling.
const mode = modeArgument ?? (!process.stdout.isTTY ? 'pipe' : process.env['CI'] ? 'static' :
↪  'interactive');
const ctx = createNodeContext();
const selectedContext = { ...ctx, mode: mode === 'interactive' ? 'interactive' as const : mode
↪  === 'static' ? 'static' as const : mode === 'accessible' ? 'accessible' as const : 'pipe'
↪  as const };
if (mode !== 'interactive') {
  process.stdout.write(present(exampleSnapshot(), selectedContext, process.stdout.columns ||
  ↪  80, args.includes('--locale=long') ? 'long' : 'en') + '\n');
} else {
  if (!process.stdin.isTTY || !process.stdout.isTTY) throw new Error('Interactive mode requires
  ↪  a terminal; use --mode=static, pipe, or accessible');
  const driver = new EffectDriver({ after: (milliseconds, callback) => { const timer =
  ↪  setTimeout(callback, milliseconds); return () => clearTimeout(timer); } });
  const shutdown = { subscribe: (stop: () => void): (() => void) => {
    process.on('SIGINT', stop); process.on('SIGTERM', stop);
    return () => { process.off('SIGINT', stop); process.off('SIGTERM', stop); };
  } };
  try { await run(createMonitorApp(selectedContext, driver, shutdown), { ctx: selectedContext
  ↪  }); }
  finally { driver.dispose(); }
}
```

Read the finalization path

The outer finalization disposes the effect driver even when the runtime exits through an error. The shutdown subscription returns a function that removes its listeners. These paths are part of the ownership explanation. They do not establish that any hypothetical subprocess extension has been stopped; that adapter is outside the example.

Follow a result without confusing its owners

A job run and a log view have different lifetimes

A job can continue while its detail is closed. A log request belongs to a selected job and a view epoch. Cancelling the latter should not destroy the former. The next workshop demonstrates that distinction with a deliberately late response.

An owner must have a closing path

An effect driver that schedules work must be disposed when the application quits. A host subscription must remove its listeners. A view request must become invalid when the view changes. When reviewing a new asynchronous feature, identify creation, use, invalidation and disposal together. If only the creation path is easy to find, the implementation is not yet easy to trust.

A boundary exercise

Read src/app.ts, src/monitor.ts and src/effects.ts in the companion. For each, write one question its tests can answer without launching a terminal and one question they cannot. Then trace the Enter route from Workshop 3 across the diagram above. You should be able to name every change of representation without inventing a direct dependency from a painted cell back into a timer.

8

Workshop 4: Results arrive after the screen changes

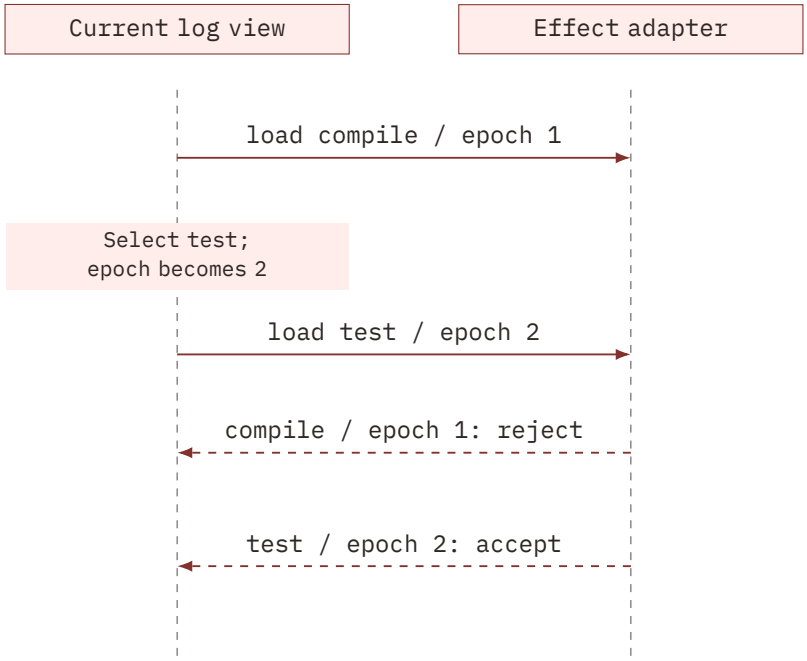
Create the race deliberately

Run npm run stage -- 04 in a terminal of at least 80 by 32 cells. Open logs for the compile job, then select the test job before the old response arrives. The adapter deliberately gives compile logs a longer delay. The first response now belongs to an earlier selection. Without an ownership check, it can replace the test job's detail pane with compile output even though the current heading still names the test job.

Identity has two dimensions

A job run carries a run identity. A view-scoped log request carries the job identity and a view epoch. Selection changes increment that epoch and request cancellation of the old log operation. Leaving the log screen also invalidates it. The monitor accepts a log completion only when the log screen is still open, the job is still selected, and the epoch still matches.

Cancellation is useful resource management. The identity check is the correctness boundary. An adapter can race cancellation, a queued message can already be on its way, or a future external transport can refuse to abort. The model still has enough information to reject a stale completion.



Trace the second interaction

The first request owns compile / epoch 1. Selecting the test job creates test / epoch 2, clears the displayed log and requests a new load. A late compile / epoch 1 completion is ignored. A matching test / epoch 2 completion can fill the pane. Closing that pane before the matching completion arrives also prevents it from reopening the screen. Run npm run stage:check – 04 to reproduce the rejection with its independent model and scheduler. The older checkpoint 04 also inspects the final model without depending on wall-clock timing.

Separate useful work from its presentation

Global jobs and local details

Changing the selected job does not cancel every job in the application. A running job belongs to the monitor’s work model; its log-detail request belongs to the current view. This difference matters. Treating every screen change as a global cancellation would destroy useful work. Treating every result as globally displayable would let old details overwrite the current screen.

The extra identities and transitions make the model larger. They also require an explicit answer to which owner remains interested in a result. That cost is preferable to making temporal ownership an accidental property of whichever callback ran last.

Close the adapter

The effect driver owns cancellation functions for scheduled work and the current log request. Disposal cancels them and prevents later emissions. The application connects quit and process-interruption messages to that lifecycle. A deterministic scheduler in the tests advances completion and cancellation without sleeping. The recorded pseudo-terminal check separately verifies that quitting and SIGTERM return the terminal’s canonical-input and echo flags on the tested host.

Break the epoch check

Run npm run exercise – stale. The faulty branch accepts the old log text into the new model. The observable failure is wrong detail for the current selection. Repair the ownership check and rerun with –repair; the detail remains empty until a matching completion arrives. Then run the timer exercise: work disposed by its owner must produce no later progress events. These are different guarantees and need separate observations.

9

Turn meaning into cells

Let geometry have one owner

A model is not a screen coordinate

The monitor identifies a selected job by its stable ID. The view decides where that job appears at the current width, height and scroll offset. This distinction allows filtering and resizing to change the visible arrangement without changing the target of an asynchronous result.

Bijou's Surface represents controlled terminal cells. In the companion, the App view returns a Surface directly. Historical design records also discuss layout trees and newer renderer contracts. Read those in their version context; the teaching example does not require a reader to adopt every intermediate representation described in the archive.

Width is measured in cells

A JavaScript string's length does not establish its painted width. A combining accent can extend an existing glyph; a wide character can occupy two cells; an emoji sequence can span several code points. The companion wraps grapheme clusters using Bijou's width calculation. A controlled width test checks the chosen cell contract, while a human terminal session checks the resulting experience in a particular environment.

The narrow view uses compact records and a scrollable body. Selection changes bring the selected job into view. Manual scrolling can inspect more content. These are related behaviors with different acceptance checks: a frame can have correct bounds while the selected item remains offscreen.

Style a role without storing decoration as truth

Keep the status as a fact

The domain stores failed, not a red escape sequence. A formatter can render that status as a word, a field or a styled component. Semantic styling means choosing appearance from the role of a piece of content. It helps a theme vary the presentation without changing what the application means.

The teaching monitor prioritizes textual state and controlled presentations. It is not a comprehensive theme-authoring demonstration. The atlas shows richer component surfaces from its explicitly identified capture version. When extending the monitor's style, preserve the plain-text fact before adding color, icons or motion.

Pay the cost consciously

A textual status consumes more width than a dot. Separate presentation functions create several outputs to maintain. A semantic style vocabulary takes work to name and keep consistent. Those costs are justified when they protect important meaning across environments. They are not an invitation to build an elaborate theme system before the application has a useful task.

Design the small view as an interaction

Choose what the person sees first

At twenty-four by eight, simultaneous visibility is scarce. Put the current context and actionable information ahead of decorative spacing. Preserve a reachable route to the error and the controls. If the required comparison will not fit, decide whether scrolling, a detail screen or an explicit minimum size best serves the task.

A smaller font in the book cannot solve a terminal layout problem. The application receives cells, and the user's terminal chooses their appearance. The Recursive specimen font is a publication choice used to show those cells consistently in print.

Test a relationship, then inspect the result

Choose a narrow frame, move selection and verify which job becomes visible. Remove color and read the error. Expand the title and check what moves. Then inspect the rendered cells. This order keeps the interesting behavior from being replaced by an attractive screenshot assertion.

A component-reading exercise

Choose the alert, modal and table specimens in the atlas. For each, identify the facts a person needs without color, the space it occupies and the input policy that the image cannot establish. Check the installed 7.2.0 exports before using a pictured API. The specimen is evidence of a capture, not a universal compatibility promise.

IV

Make it dependable

Make the promises observable

A readable frame is one observation. It does not establish that a hidden page cannot receive input or that a disposed task cannot emit another result. This Part puts geometry, lifecycle and meaning under independent checks.

What you will be able to do

Exercise the constrained presentation, run the repaired fixtures, and use the troubleshooting sequence to reduce a failure to the smallest relevant boundary. Keep human usability questions explicit.

10

Workshop 5: When the terminal fights back

Reduce the available space

Run npm run stage – 05, then resize the interactive monitor to twenty-four columns and eight rows. Below forty-two columns, its presentation changes from a table to compact job records. The renderer wraps the content into cells, keeps a small heading region, and provides a scrollable body with a footer. Moving selection brings the selected job into the visible body; j and k let the reader inspect more content. The model still owns job identities and selection while the geometry changes.

Do not confuse clipping with usability

A Surface bounded to its allocated rectangle can be geometrically correct while hiding the only cancellation instruction. The companion tests dimensions down to one cell to catch allocation and width errors. Its practical small-viewport target is twenty-four by eight. The one-cell checks establish bounds, not a claim that the complete task is usable in that space. A product should declare its minimum useful geometry and provide an understandable recovery path below it.

Terminal cells are a constraint

String length is not cell width. A wide character can occupy two cells, a combining accent can share a cell, and an emoji can contain several code points. The companion’s cell-wrapping helper walks grapheme clusters and uses Bijou’s width calculation. Its tests include these cases at controlled dimensions. A cluster too wide for a one-cell allocation receives a visible placeholder. That is a deliberate loss of shape at an extreme bound, not a silent claim that every character fits.

The cost is another layout pass and a dependency on the terminal’s width conventions. Actual emulators and fonts can disagree about edge cases. Controlled cell tests establish the renderer’s chosen contract; they do not replace checking the terminals supported by the product.

Remove presentation assumptions

Long copy and no color

Run the final stage with –mode=static –locale=long, or use the original checkpoint 05 for its fixed narrow fixture. Its long-copy fixture exposes assumptions hidden by short English labels. The fixture is not a localization catalog: it changes the title to exercise expansion while the rest of the application remains English. A real localization exercise would move complete messages, counts and direction into an explicit catalog and test their effect on focus and geometry. Translating a few words is not the same acceptance claim.

Run the static mode with NO_COLOR=1. A failed job must still have a word identifying its state and an explanation. Redirect output and inspect the TSV. A consumer should not have to strip interactive cursor control codes before reading the data. Force interactive mode without a TTY and observe the explicit error rather than an application pretending that raw keyboard input is available.

Interrupt at an inconvenient time

Start a job and close the application before it completes. Scheduled callbacks belong to an owner that must dispose of them. The timer exercise makes a missing disposal observable; the separate pseudo-terminal script tests terminal restoration after SIGTERM. For a real process runner, interruption also needs a policy for child processes, partial output and a meaningful exit status. The current simulated adapter does not establish those guarantees.

Choose an honest intermediate result

The small-layout implementation retains statuses, numeric progress, errors, selection and controls through compact text and scrolling. It gives up simultaneous visibility of every column. That tradeoff should be judged against the task. If an operator must compare hundreds of jobs at once, a different interaction or a larger minimum viewport may be appropriate. A framework’s ability to paint a small rectangle does not make every workflow fit inside it.

11

Workshop 6: How do we know it works?

Match each claim to an observation

Run npm run verify. The pinned companion edition typechecks and passes 44 tests across eight files, including the five independent stages and the final model/effect and presentation/application suites. The table below names the purpose of the evidence. The pseudo-terminal checks are retained separately because they exercise a real process and terminal device rather than the deterministic model.

Level	Observation	Boundary
Application state	Inputs produce the expected immutable transitions	Does not run a terminal
Asynchronous work	Success, failure, cancellation, stale results and disposal	Scheduled fixture; no subprocesses
Layout	Controlled dimensions and Unicode cell bounds	Bounds alone do not prove task usability
Rendering	Dialog cells and mode-specific output for a controlled model	A single captured state
User experience	Keyboard routes; separate PTY keyboard, resize and shutdown checks	No human screen-reader or broad emulator study

Rows 1–5. Actual Bijou table component.

State and effects

A state test starts a job, applies progress and checks the result. It also sends a late completion after cancellation and restart. This proves that the transition rejects the old run under those inputs. The scheduler test advances known delays, observes success and failure, cancels work and disposes the driver. It establishes which messages are emitted, including the absence of later messages after disposal.

Those tests do not establish the behavior of an operating-system process runner. The example has none. Replacing the scheduled fixture with subprocesses introduces buffering, exit codes, signal delivery and process-tree ownership that require their own adapter checks.

Geometry and cells

The layout test checks controlled widths and heights, including wide characters, combining text and an emoji sequence. The routing test supplies actual Bijou framework messages to the App and inspects the Surface produced for cancellation. These checks connect input, state and rendering. They still cannot tell whether a human notices the prompt, understands a failure message or can navigate effectively with a particular assistive tool.

Read failure as information

Make the broken state reproducible

The four exercise fixtures fail intentionally: modal input leaks, a stale result replaces current detail, a colored mark loses meaning, and work emits after closure. Each has a small command, a fixed starting state and an observable success criterion. The repaired path uses the production implementation. Default CI runs the ordinary suite and the repaired exercises; it does not mistake an intentionally broken teaching fixture for a regression in the final application.

Avoid convenient evidence substitutions

A typecheck does not establish a temporal property. A screenshot does not establish input ownership. A cell-bound test does not establish that every control remains reachable. A keyboard-routing test does not establish how a screen reader announces the application. Keep the attractive artifact and the behavioral evidence together while naming what each establishes.

Read the decisions and their costs

Semantic styling

The problem is a useful state disappearing when color disappears. A tempting shortcut is to make red mean failure by itself. The chosen starting point is textual state with optional visual reinforcement. This spends columns and can feel less compact. It preserves a fact that every mode can carry. Semantic tokens become useful when styling is added: the role remains “error” while a theme chooses its appearance.

Explicit ownership

The problem is an action reaching a page behind its dialog or a result reaching a screen that no longer owns it. Global shortcuts and unqualified callbacks look simple until those owners diverge. The chosen model makes input boundaries, run identities and view epochs explicit. The cost is more

state and more transitions to test. The benefit is an ownership claim that can be examined before a convincing frame hides the defect.

Meaning separate from presentation

The problem is a consumer needing facts that were stored only as a rendered string. Keeping an immutable model and several formatters adds code paths and compatibility obligations. It also lets the same failed job appear in a table, a CI snapshot, TSV and explicit accessible prose without reconstructing its identity from decoration. Every added presentation must preserve the task’s agreed facts; sharing a model does not make a formatter correct automatically.

The separate answer key explains each repair and why a plausible shortcut still fails. Use it after attempting the faulty fixture. The unfamiliar-reader protocol is prepared; a participant session is still pending, and no human results are reported.

12

Troubleshoot from the symptom

Keep a useful failure record

Fix the environment before changing the explanation

Record the companion commit, Node and npm versions, command, presentation mode, viewport, relevant terminal settings, expected behavior and observed behavior. Save the raw output when possible. The reference commit and package lock make it possible to ask whether another person sees the same failure.

First reproduce with the pinned edition and its existing checks. If a dependency upgrade is already involved, keep the old and candidate environments distinct. A repair that also replaces the runtime, lockfile and input fixture can hide which change mattered.

The job starts when a dialog is open

Inspect routing and transition boundaries

Start a job, open its cancellation confirmation, and send movement or another start intention. Observe selection and effects before looking at the overlay. The production model refuses the blocked intentions. Enter inside confirmation is translated into a cancellation intention rather than an ordinary start.

Use the modal exercise to reproduce a boundary violation. If the model is correct but actual keys still behave differently, inspect the App's translation of the framework key message. If the model accepts the wrong event from any source, repair the transition contract first. A stronger border or higher drawing layer cannot fix input ownership.

Logs show the wrong job

Inspect identity and lifetime

Record the selected job, screen and epoch when requesting logs and when accepting them. Use checkpoint 04 to observe an old completion rejected after

selection changes. A log response must match all three conditions: the log screen remains open, the job is selected, and the epoch matches.

A response can arrive after its cancellation was requested. That makes cancellation alone insufficient as the correctness boundary. If the wrong result is accepted, compare its ownership fields before changing its delay. Increasing the delay only changes how often the race appears.

Rows wrap badly or disappear

Separate bounds from visibility

Reproduce at a fixed width and height. Compare grapheme width with the intended allocation rather than string length. Use the controlled Unicode fixture. Next inspect whether selection is in the scrollable body and whether the footer remains available. A Surface with the right dimensions can still contain the wrong crop.

If the controlled cells are correct but a particular emulator displays them differently, retain the exact text sequence, terminal version and font. That is a different reproduction from a layout calculation failure. The book's glyph projection does not reproduce every emulator's shaping and width convention.

A pipe contains unexpected text

Inspect the producer before the serializer

Use node –import tsx src/main.ts –mode=pipe > jobs.tsv or the documented silent npm invocation. An npm script banner is output from the command launcher, not a field emitted by the monitor's serializer. Inspect exact tab-separated fields, including empty trailing error fields.

The pipe contract has four columns: ID, textual status, numeric progress and error. It contains no interactive help or cursor control. If those facts fail, reduce the observation to the formatter with a fixed snapshot. A clean-looking terminal preview does not establish that tabs and empty fields survived redirection.

The terminal remains in an awkward state

Follow shutdown through the owning process

Distinguish normal quit, Ctrl+C and an external SIGTERM. The host adapter subscribes shutdown signals, the App disposes its driver and returns the quit command, and the runtime closes its terminal session. The companion also disposes the driver in a finalization path. Its recorded pseudo-terminal checks verify restoration on the tested host.

If a real child process is involved in an extension, add that process tree to the ownership investigation. The simulated driver's cleanup cannot establish anything about an operating-system child it never created. Preserve process exit status and terminal flags as separate observations.

Everything passes, but the application is hard to use

Ask a human question with a human observation

Tests can show that a selected relationship is present in accessible text. They cannot tell whether a person hears it at a useful moment, finds the action or understands the error. Use keyboard-only and assistive-tool sessions with a concrete task. Record the tool and environment, what was attempted, what became confusing and what the person could complete.

The companion has no recorded human screen-reader study. That is an evidence boundary to improve, not a result to infer from 44 passing tests. A useful follow-up names the missing observation and its acceptance criteria.

Run a bounded usability exercise

Ask a participant to identify the failed job and explain its failure from the accessible snapshot, then do the same with the color-free static output. Record the exact fixture, assistive tool, reading order and the participant's answer. For interaction, separately ask someone using only the keyboard to start a job, inspect logs, open cancellation, dismiss it and quit. Observe where they hesitate and whether they can explain the final state. These are proposed reader exercises; no participant results are claimed for this edition.

The accessible CLI snapshot exits after output, so it cannot establish an interactive screen-reader workflow. Such a workflow requires an implemented interaction contract and observation with the intended tool. Keeping those two exercises separate prevents a successful reading of a static report from becoming an unsupported claim about a live application.

Ship and maintain it

Keep the example reproducible

A useful application has a life after its first successful demonstration. These chapters record the edition, inspect its source package and walk through a deliberately bounded maintenance change.

What you will be able to do

You should be able to reproduce the published checkpoint, describe what its evidence covers, and propose an upgrade without silently changing the reference edition.

13

Workshop 7: Ship a reproducible example

Pin the thing a reader can actually run

The companion lives in its own public repository: https://github.com/flyingrobots/bijou-job-monitor. This edition uses tag book-v2 and the exact commit recorded on the version page. The package lock pins published Bijou 7.2.0 and the supporting toolchain. The records in the companion archive discuss several versions and aspirations; they are not the dependency manifest for this application.

Reproduce a build stage

Clone the repository, check out the recorded commit or edition tag, and run npm ci. Run the build stage for the workshop, then its stage check. Five independent snapshots cover the model, four presentations, input, asynchronous ownership and constrained geometry. Each directory owns its modules and shares the root dependency lock. The adjacent patches are actual source diffs. The original checkpoint commands separately inspect selected behaviors of the finished modules.

The published book-v2 tag fixes the whole route to the recorded commit. The prior book-v1 remains available. When making an incompatible change, publish another edition reference; a reader must be able to recover the source that produced the explanation. The answer key and walkthrough protocol ship with the source. The blank participant form remains unfilled until an actual session occurs.

Verify the distributed source

Run npm pack –dry-run and inspect the listed files. The code package includes the full Apache 2.0 license, its NOTICE, source-file copyright statements, checkpoints and exercises. Git checkout also retains the package lock. Dependencies retain their own applicable notices if they are redistributed with a package. The copyright page in the book does not replace these files.

Keep claims attached to their environment

An environment record has limits

The version page identifies Node, npm, the companion commit, the published Bijou release and the locally tested host. The companion's VERIFICATION.json records 44 passing tests and the pseudo-terminal observations. GitHub Actions runs the pinned checks on Linux; its current result belongs to that run and should be read from the repository. No human screen-reader session or broad terminal-emulator matrix was performed for this edition. Those are open acceptance questions rather than implied guarantees.

Make corrections findable

Use the companion's issue tracker for book-example errata. Include the workshop, edition commit, command, environment, expected result and observed result. A correction becomes more useful when another reader can reproduce it. Keep the original edition available while a later edition explains the repair and its compatibility effect.

Let the book be a working specimen

The monitor already lets readers start jobs, switch to logs, change selection, filter failures and resize the view. The explicit CLI modes let them inspect the same fixture as static text, TSV or accessible prose. Its tests expose state and cells without requiring an image. The printed explanations remain complete; the running application supplies another way to examine their claims.

A future inspector could make model snapshots and layout rectangles visible inside the application. That capability is not implemented in this checkpoint. It should be added when it answers a concrete learning question and arrives with its own evidence. The useful flourish is the connection between the explanation and a working surface, not an unverified menu promising every debugging feature.

14

Maintain the promise after release

Treat the reference edition as a reference

Keep the starting point available

The book pins the companion to an immutable commit. Reproduce that commit before making a change. Work on a separate branch and retain the old edition tag. This makes a disagreement about behavior answerable: run the same command against both candidates and compare the observation that matters.

A dependency upgrade is its own change to the environment. Inspect the candidate release's actual exports and migration information, then run the existing evidence against the new lockfile. Do not rename the old tag to make the book appear current. A new edition should record its own commit and explain compatibility differences.

Walk a concrete maintenance change

The bug: cancellation overwrites completion

Consider this report: "I open cancellation confirmation while compile is running. Compile completes before I press Enter. Confirming then makes the job look cancelled." This is a temporal defect. A screenshot of either final state will not explain the ordering that produced it.

The pinned companion already contains the regression check and repair for this case. Inspect the test named completion during confirmation cannot be turned into cancellation in test/monitor.test.ts. It starts a run, opens confirmation, applies a matching completion, then confirms. The expected observations are a successful job, a closed dialog and no cancellation effect.

Why the repair belongs in the model

The production confirm-cancel transition checks the selected job's current state. If confirmation remains open after the job stops running, it closes the dialog without changing the completed job or emitting an abort. This

protects callers beyond the keyboard adapter as well as the visible interface.

A repair that only disables a button can miss keyboard input or a delayed event. A repair that cancels on dialog opening changes the promised interaction: Escape would no longer leave the job running. The model repair preserves the original promise while accounting for the later fact.

Reproduce the failure on a disposable branch

As a reader exercise, temporarily remove the non-running guard from that transition. Run npm test and observe the named regression fail. Restore the guard and rerun. The acceptance criterion concerns state and emitted effects under a fixed sequence; it does not depend on making a human press Enter at just the right millisecond.

This exercise deliberately modifies a copy of the example. The printed production file and pinned commit remain the reference. Do not commit the broken variant as a new checkpoint or replace the edition’s passing verification record with a result from another tree.

Choose evidence in proportion to the change

A transition repair

For the cancellation race, the deterministic state sequence is the central regression. Existing routing and effect tests establish adjacent contracts. A focused interactive check can confirm that the explanation and prompt still make sense. A new renderer benchmark would not establish the repaired temporal property.

A dependency upgrade

An upgrade can affect App messages, Surface behavior, adapters and terminal cleanup. Run typechecking, model/effect tests, controlled rendering checks, CLI output checks and the relevant pseudo-terminal observations. Review the mode outputs for compatibility. Test human workflows where the changed behavior can affect them, and record what was actually observed.

If a newer component promises a richer view, add it only after verifying that it belongs to the target release. The archive and atlas are useful sources of questions; their presence in the publication does not certify the upgrade.

Keep corrections traceable

An erratum needs a reproducible boundary

A useful report includes the edition commit, command and environment, the expected result, actual result and smallest known reproduction. Link its fix to the test or observation that establishes the repaired promise. If the explanation was wrong but the code was correct, amend the prose and identify the affected edition.

Leave a useful next application

A real process adapter, complete localization or an integrated inspector can each be a coherent extension. Give one extension an observable outcome, an explicit boundary and a working intermediate result. Preserve the existing modes and ownership contracts as it grows. The application should become more useful with each change, and the next reader should still be able to explain why it works.

Reference

These reference chapters support application work. Component artwork is actual rendered output from the capture checkout, identified as 8.0.0-rc.1. It is a visual reference, not a claim that every pictured component or API is available in the teaching application’s pinned 7.2.0 release. The renderer gallery identifies its separate 7.1.0 source. Consult a release’s exported types before using an unfamiliar component.

A

The component cell atlas

These are deterministic vector projections of actual DOGFOOD component stories at time zero, using the shipped BIJOU_DARK theme and interactive profile. Every glyph, position, colour and modifier comes from the rendered Surface. The terminal font is explicitly Recursive Mono with DejaVu Sans symbols. They are not photographs of an emulator. No component has been redrawn by eye.

alert()

Persistent in-flow status block for success, warning, error, and informational state that should remain part of the page instead of floating above it.



Story: alert; variant: success; 40 × 3 cells.

badge()

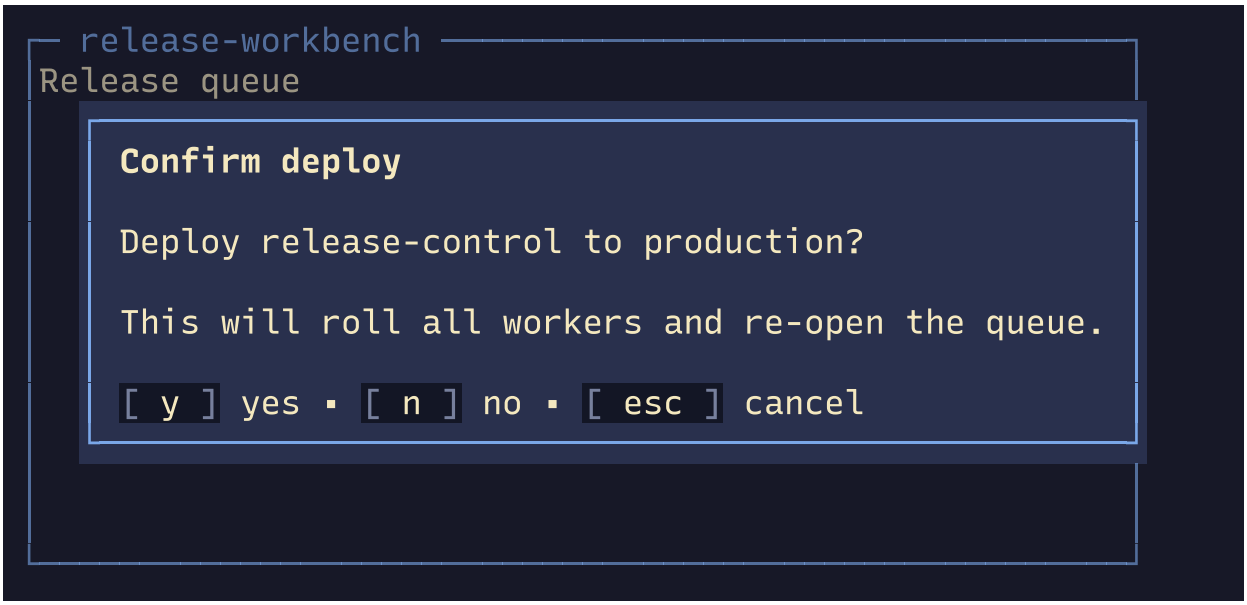
Compact inline status label for states that belong beside another object instead of becoming their own block.



Story: badge; variant: service-health; 35 × 5 cells.

modal()

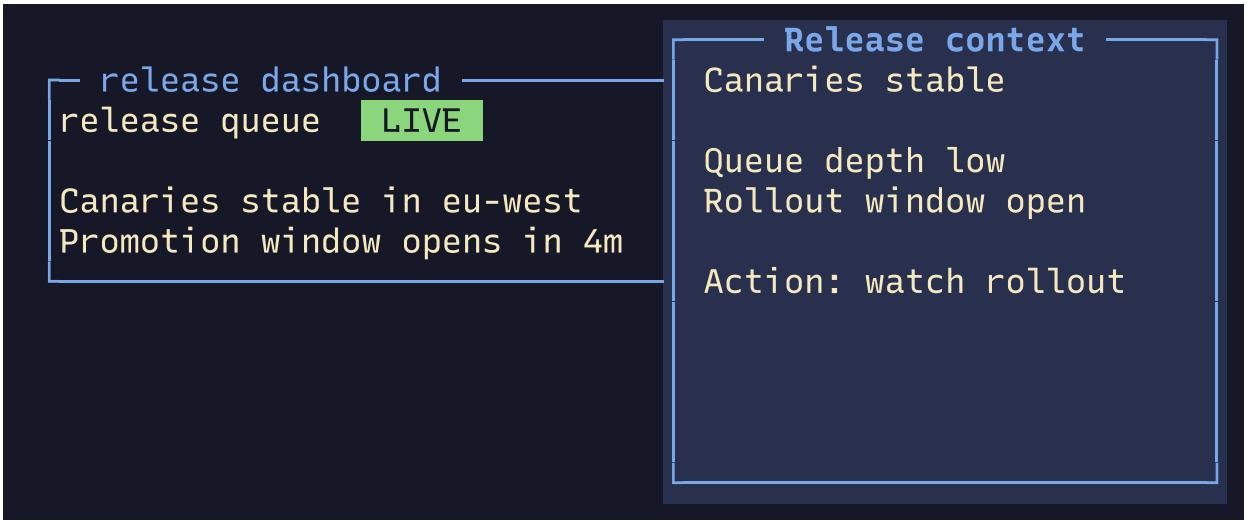
Centered blocking dialog for decisions that should temporarily own attention and input inside a TUI surface.



Story: modal; variant: confirm; 60 × 14 cells.

drawer()

Anchored supplemental panel for sidecar context that should remain related to the current surface without blocking the whole task.



Story: drawer; variant: supplemental-right; 60 × 12 cells.

tooltip()

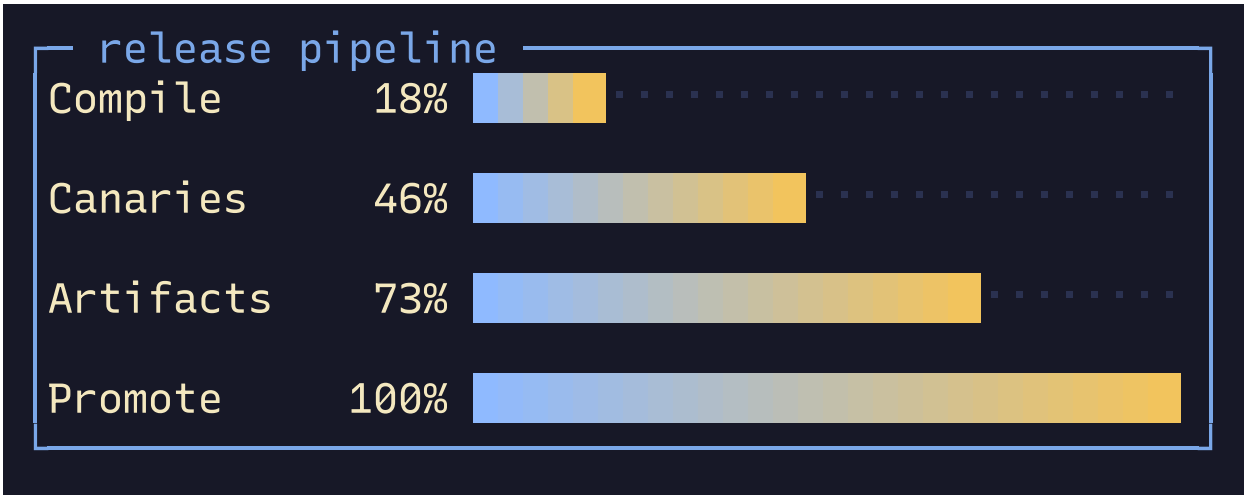
Short local explanation positioned near the thing it describes, for cases where the meaning is helpful but not part of the document flow.



Story: tooltip; variant: local-explanation; 60 × 10 cells.

progressBar()

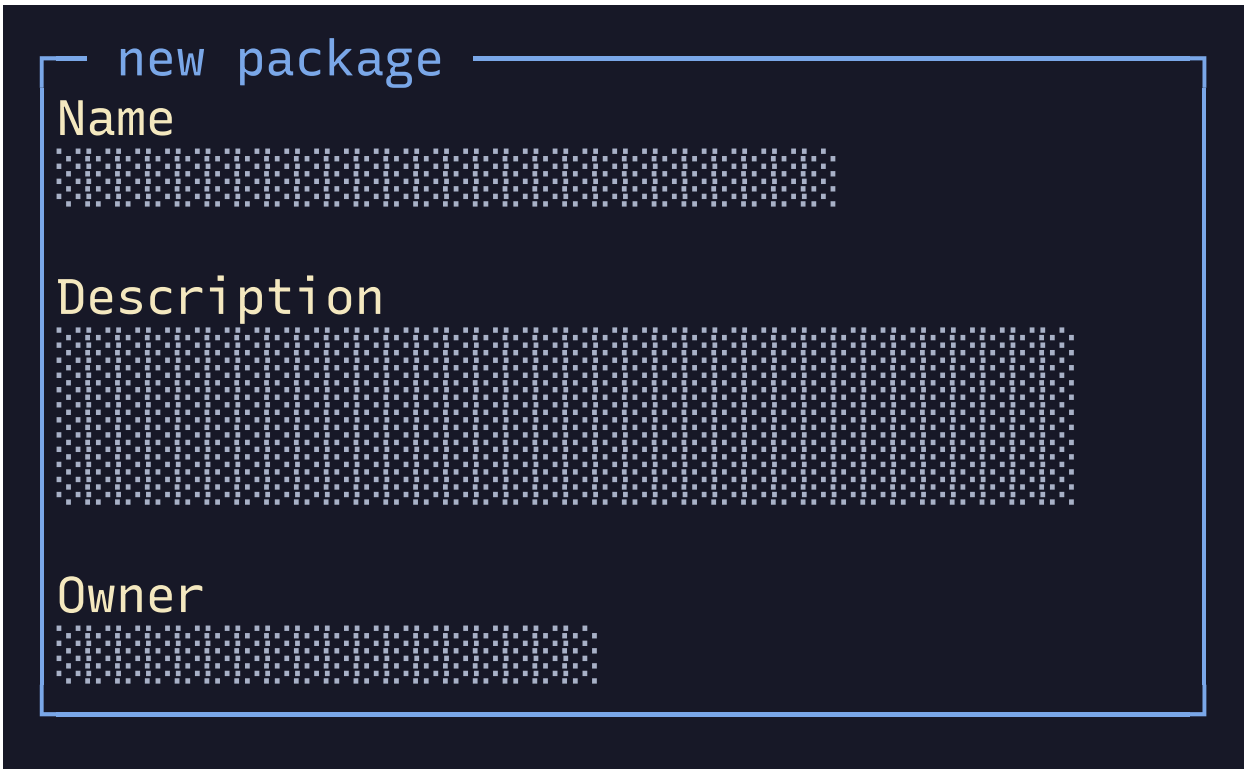
Honest determinate progress indicator for work that can report a real completion percentage, with related support for indeterminate activity cues.



Story: progress-bar; variant: release-milestones; 48 × 9 cells.

skeleton()

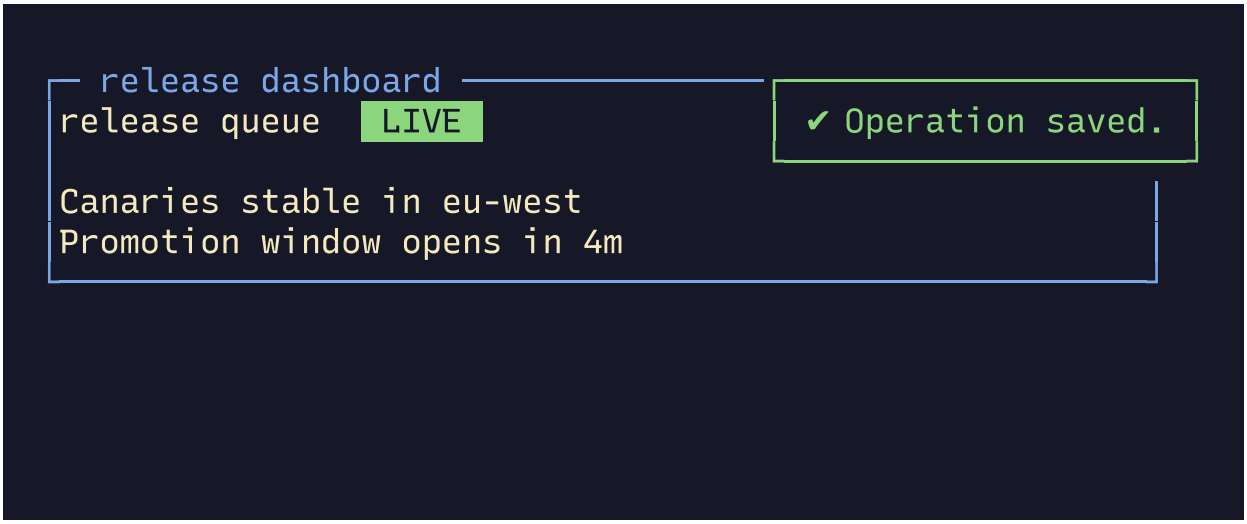
Shape-preserving loading placeholder for short-lived uncertainty when you know the information layout but not the data yet.



Story: skeleton; variant: form-shell; 40 × 12 cells.

toast()

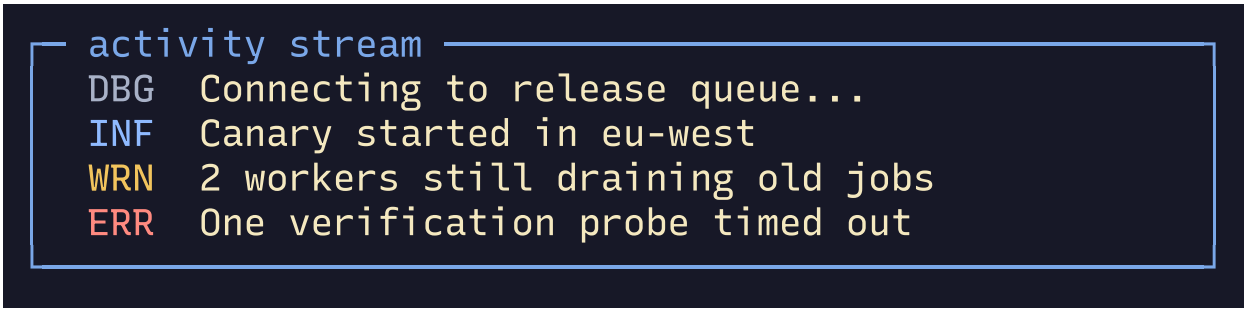
Low-level transient overlay primitive for one directly composed, anchored notification when the full notification system would be too heavy.



Story: toast; variant: saved-top-right; 60 × 12 cells.

log()

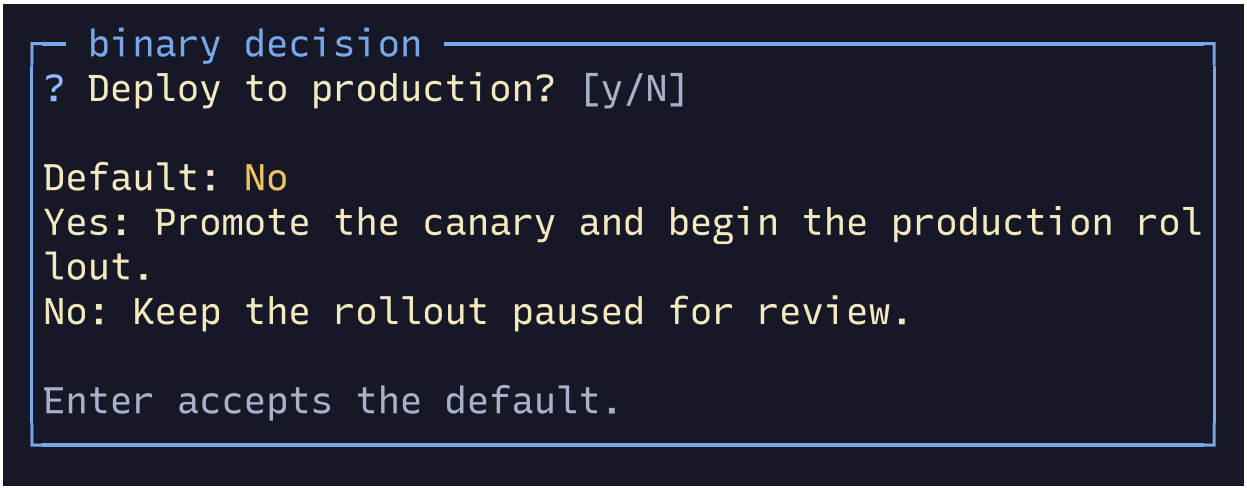
Chronological activity-stream primitive for accumulating operational events with explicit severity and optional timestamps.



Story: log; variant: release-events; 54 × 6 cells.

confirm()

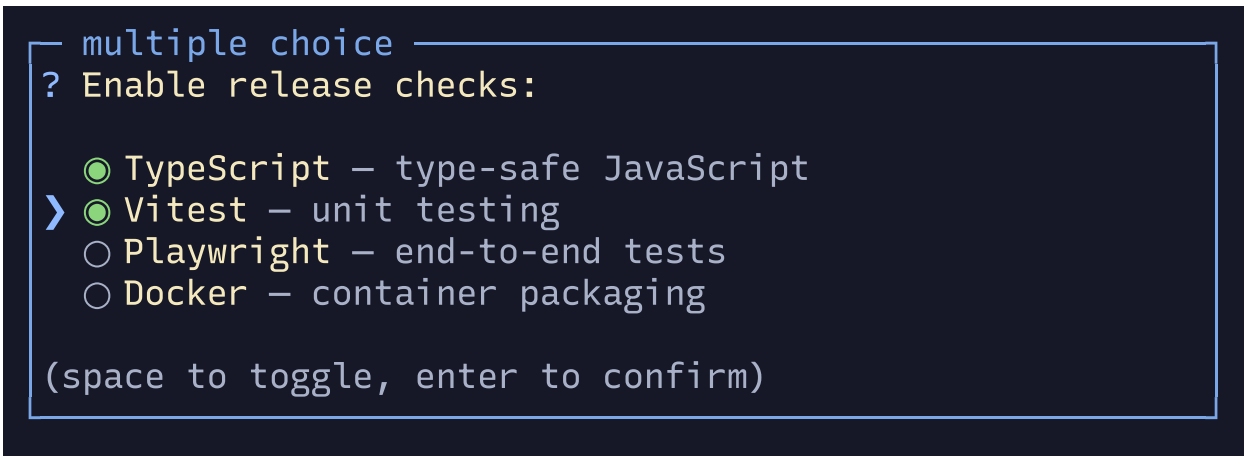
Binary decision prompt for real yes/no choices where the consequence of accepting or declining should stay explicit.



Story: confirm; variant: deploy-gate; 54 × 10 cells.

multiselect()

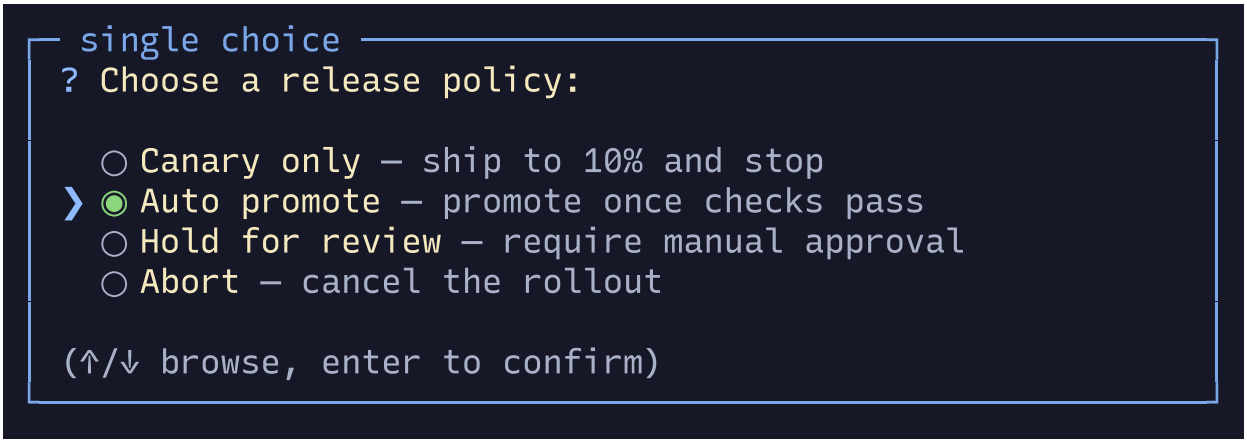
Set-selection prompt for building a durable group of choices with keyboard toggling and truthful numbered fallbacks.



Story: multiselect; variant: release-stack; 58 × 10 cells.

select() / filter()

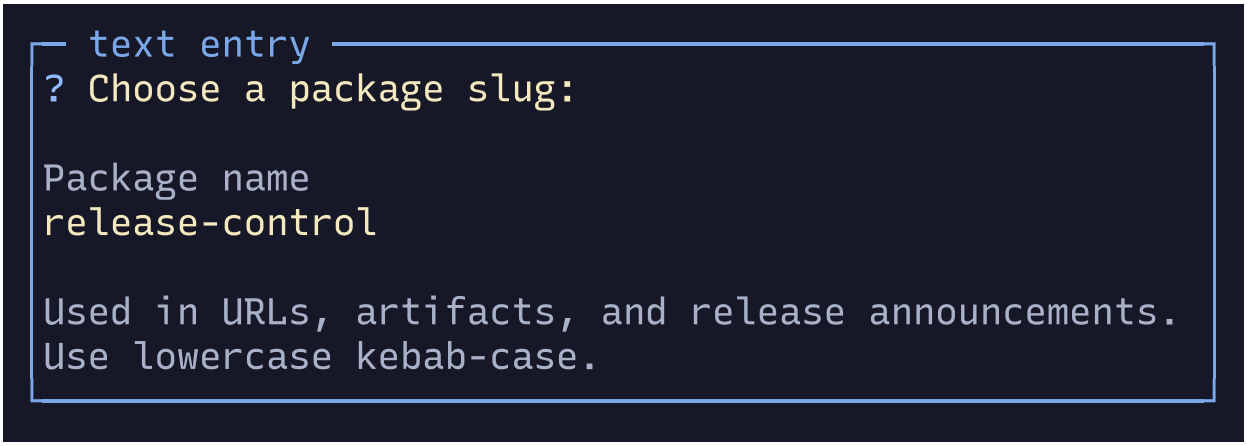
Single-choice prompt family for choosing one lasting value, either from a visible list or from a searchable narrowed list.



Story: select; variant: release-policy; 60 × 10 cells.

input() / textarea()

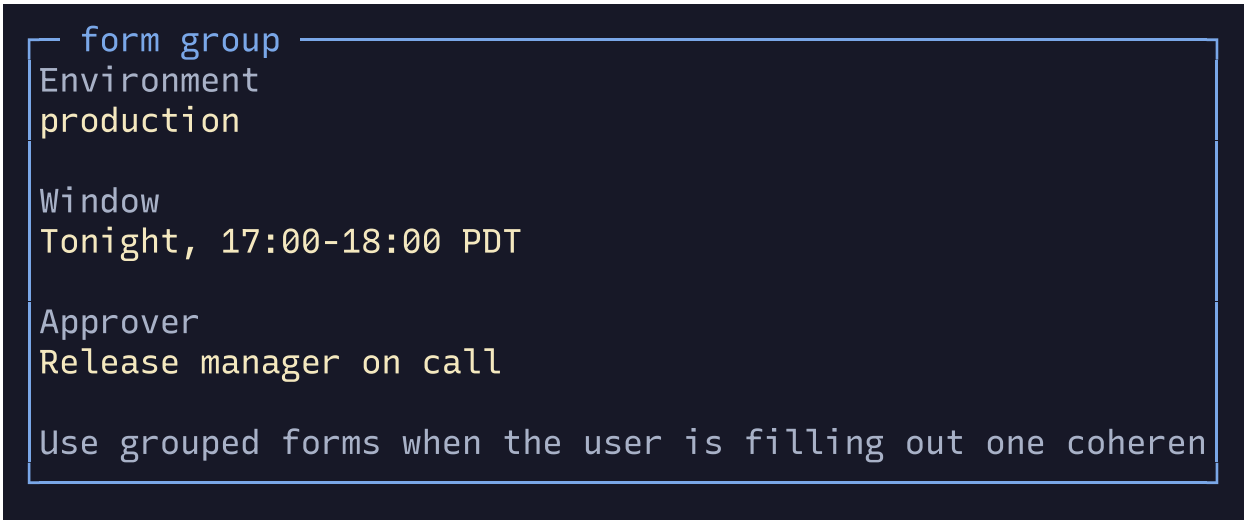
Text-entry family for collecting freeform user input, from short single-line identifiers to longer multiline notes that still need honest lowering.



Story: text-entry; variant: service-name; 54 × 9 cells.

group() / wizard()

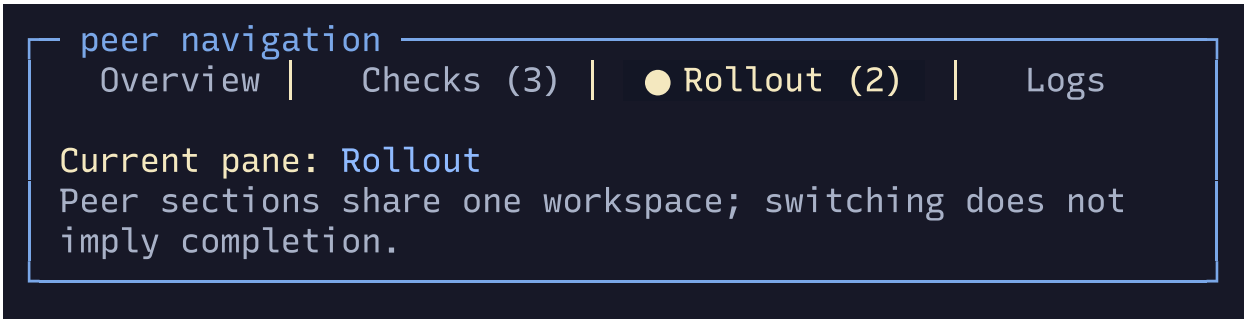
Grouped and staged form family for related inputs that belong together, whether they happen in one focused section or across explicit steps.



Story: group-wizard; variant: deploy-group; 60 × 12 cells.

tabs()

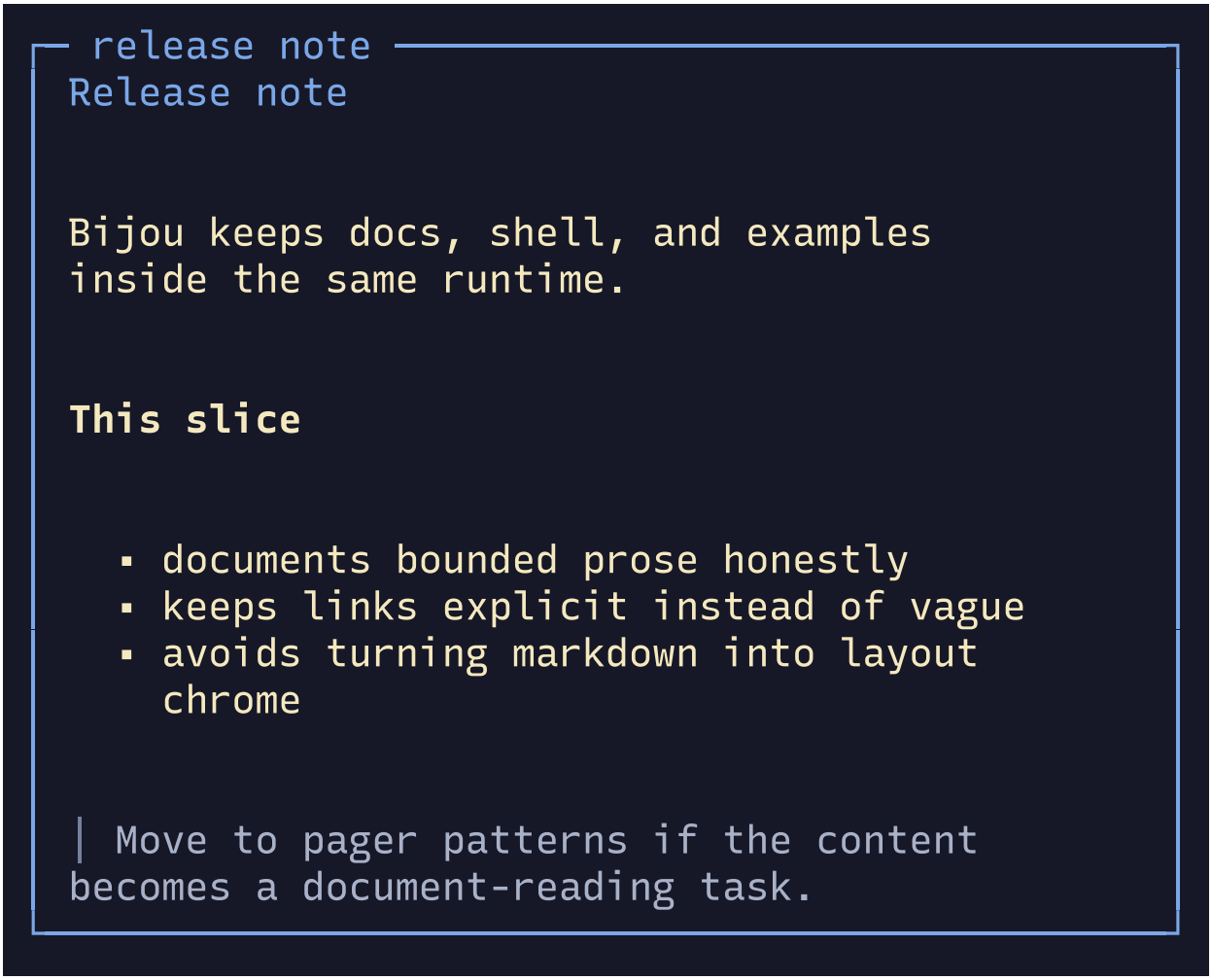
Peer-navigation strip for switching between sibling sections that share one workspace and one conceptual level.



Story: tabs; variant: release-workbench; 60 × 7 cells.

markdown()

Bounded structured prose renderer for help, release notes, and reference text that should stay honest across rich, pipe, and accessible modes.



Story: markdown; variant: release-note; 50 × 20 cells.

hyperlink()

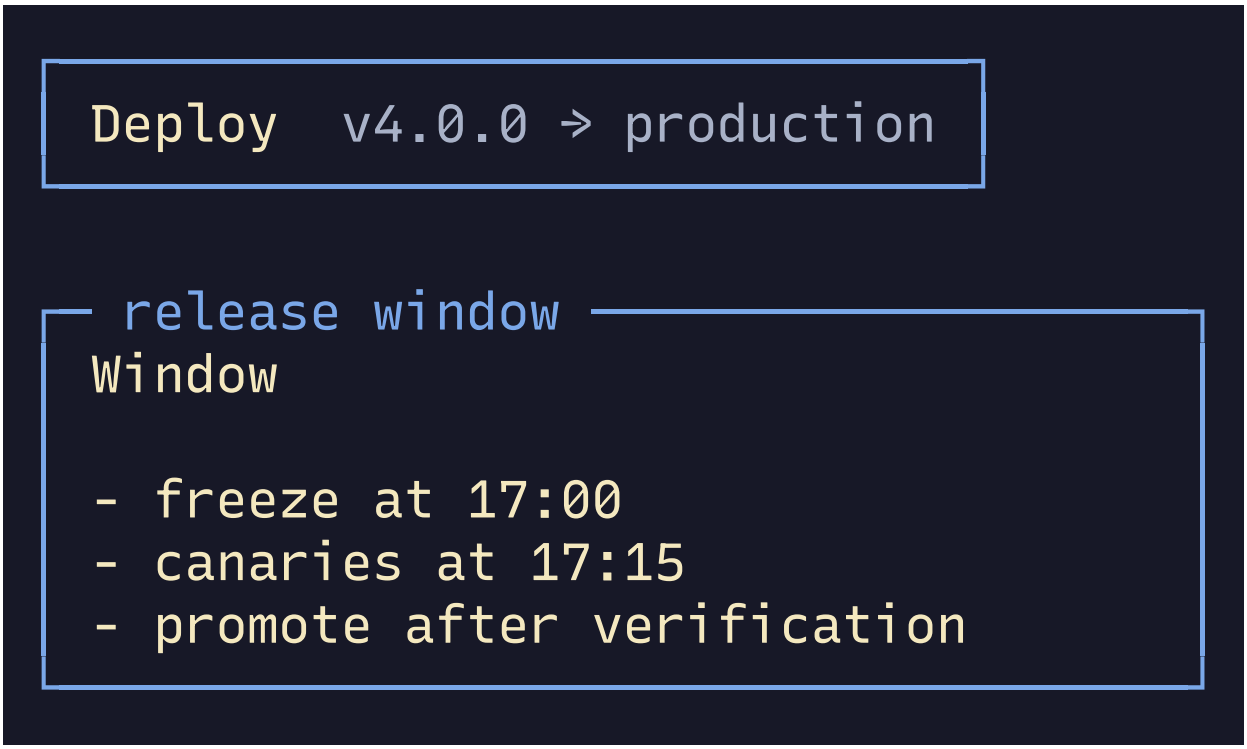
Explicit destination primitive for links that should remain meaningful whether or not the terminal supports clickable OSC 8 output.



Story: hyperlink; variant: explicit-destinations; 58 × 5 cells.

box()

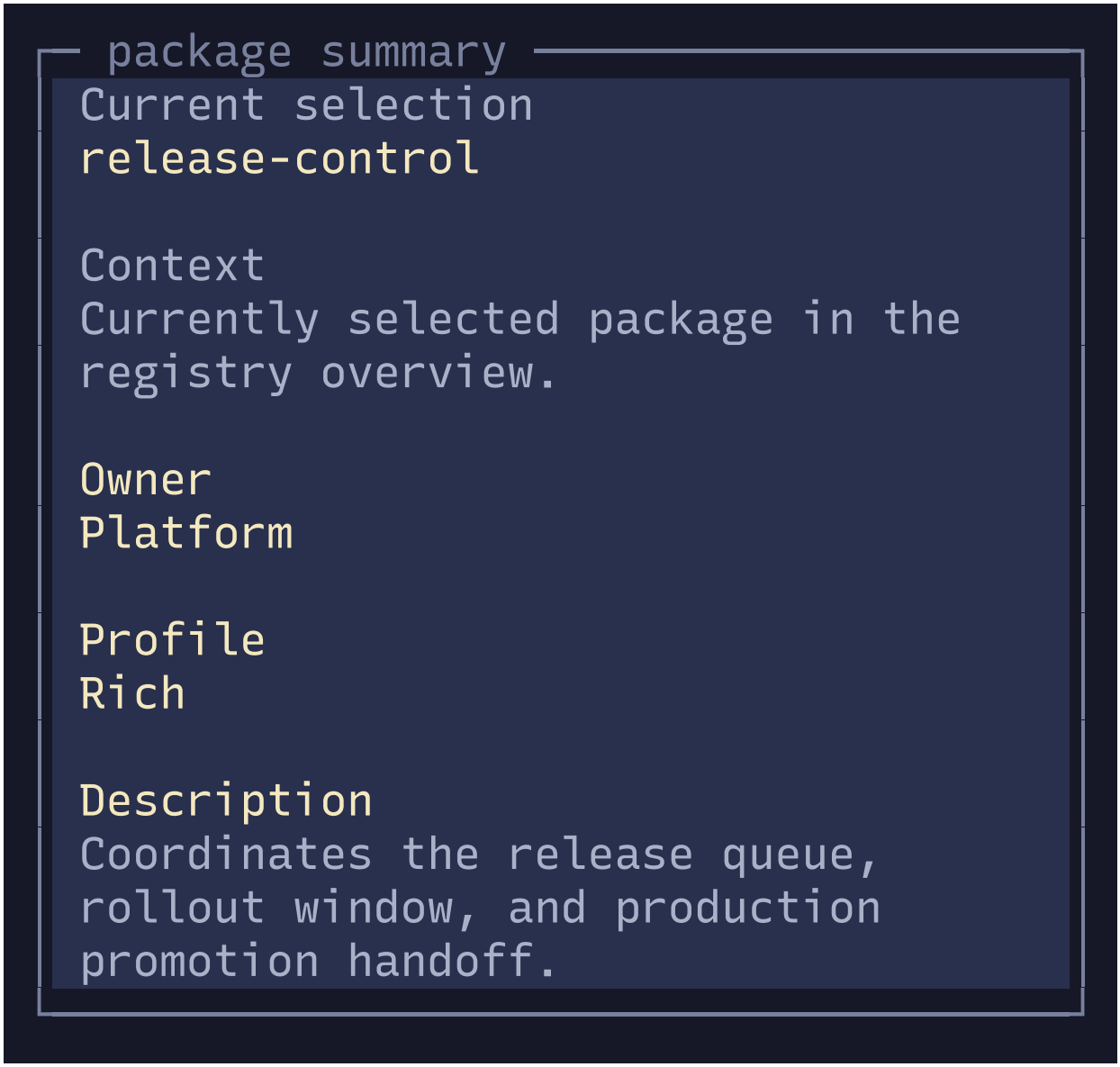
Canonical containment primitive for grouped content and compact titled panels, with ‘headerBox()‘ and surface companions for richer layout composition.



Story: box; variant: header-detail; 38 × 11 cells.

inspector()

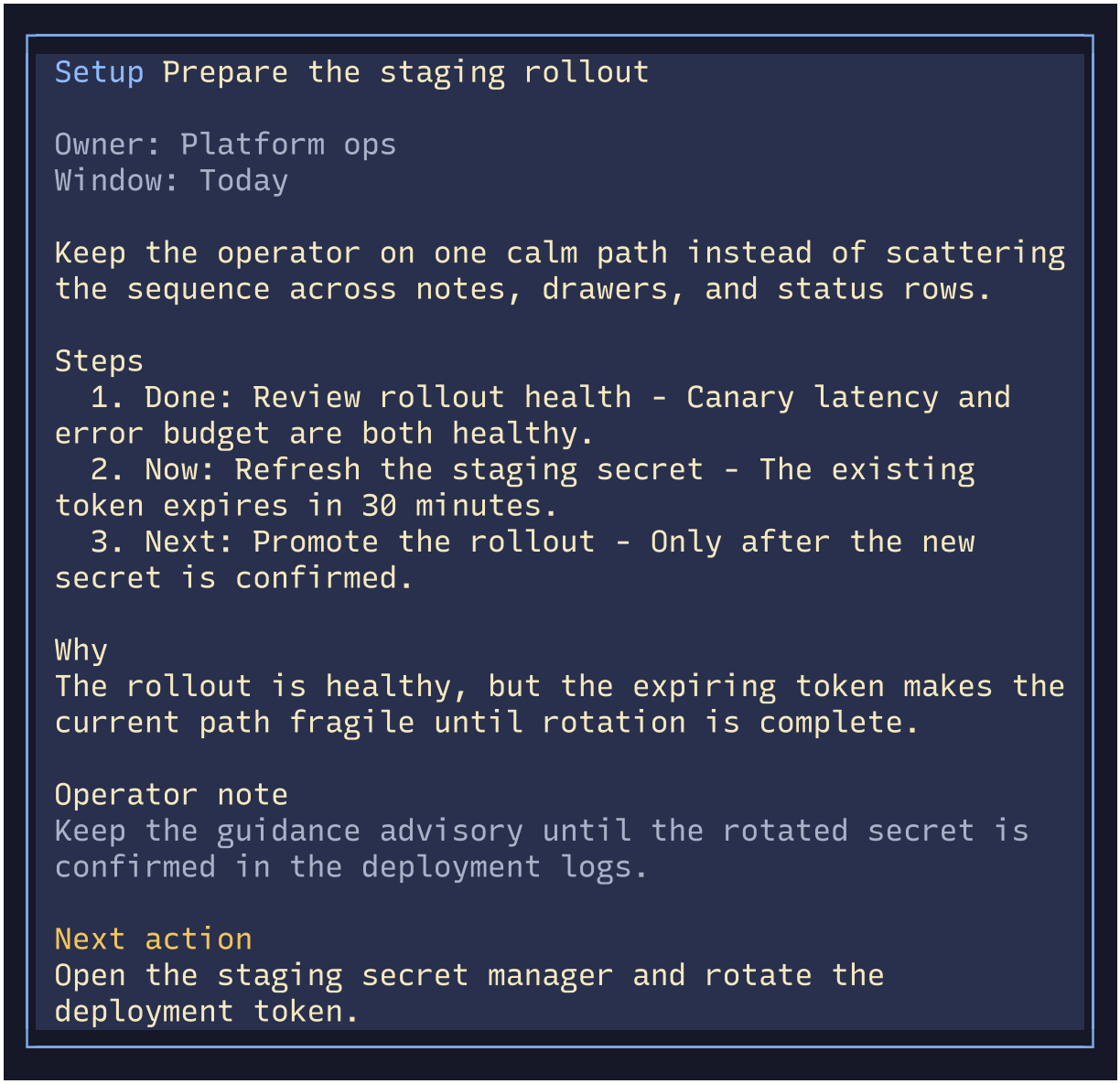
Canonical side-panel summary surface for the currently selected thing, keeping one obvious active value and calmer supporting sections nearby.



Story: inspector; variant: package-summary; 40 × 19 cells.

guidedFlow()

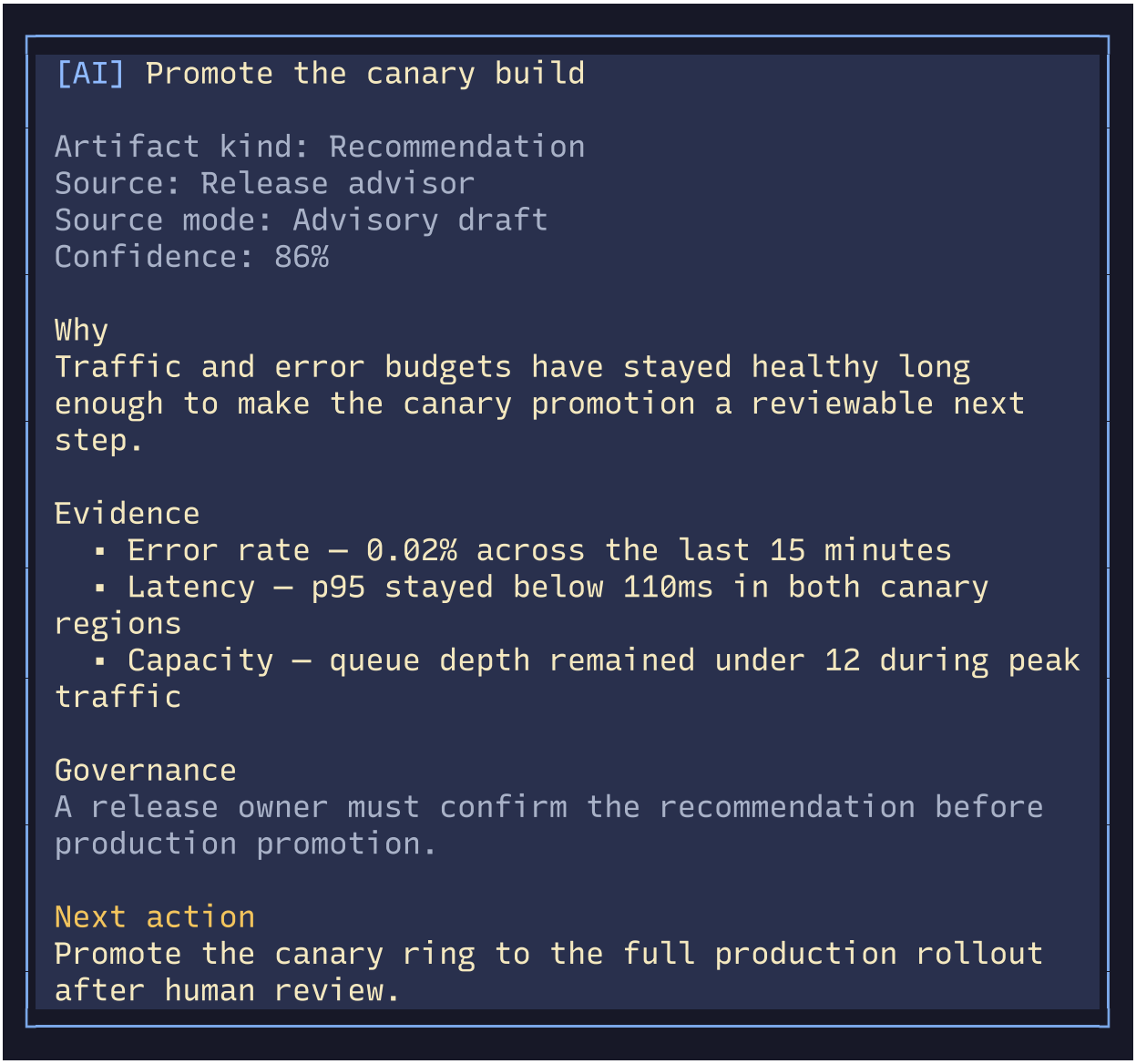
Reusable block for calmer multi-step assistance surfaces with explicit sections and one obvious next action.



Story: guided-flow; variant: staging-rollout; 60 × 29 cells.

explainability()

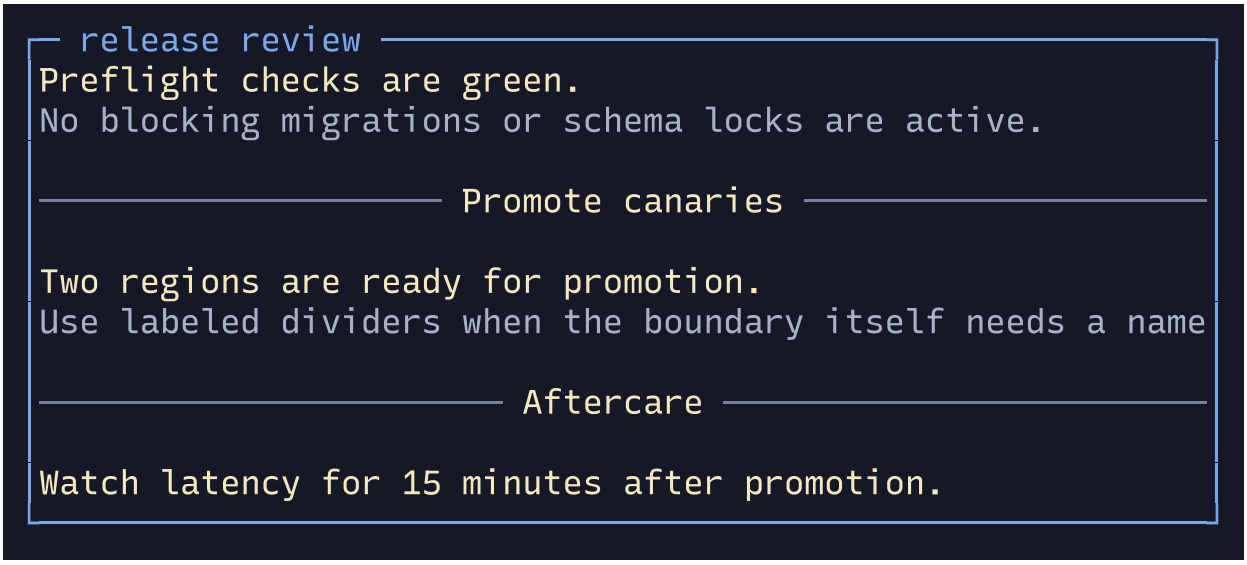
Calm guided recommendation surface for AI-mediated or machine-assisted output that must make provenance, evidence, and next action explicit.



Story: explainability; variant: rollout-recommendation; 60 × 28 cells.

separator() / separatorSurface()

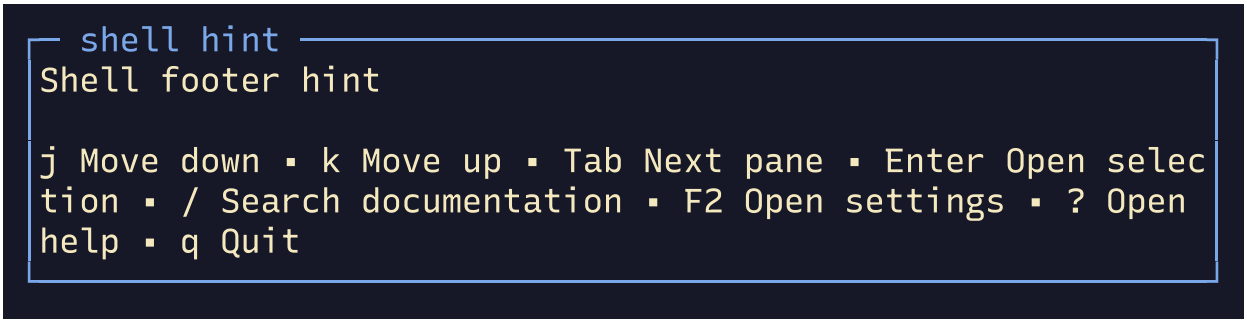
Section-boundary primitive for marking real transitions without promoting every boundary into a boxed panel.



Story: separator; variant: labeled-breaks; 60 × 13 cells.

helpView() / helpShortSurface()

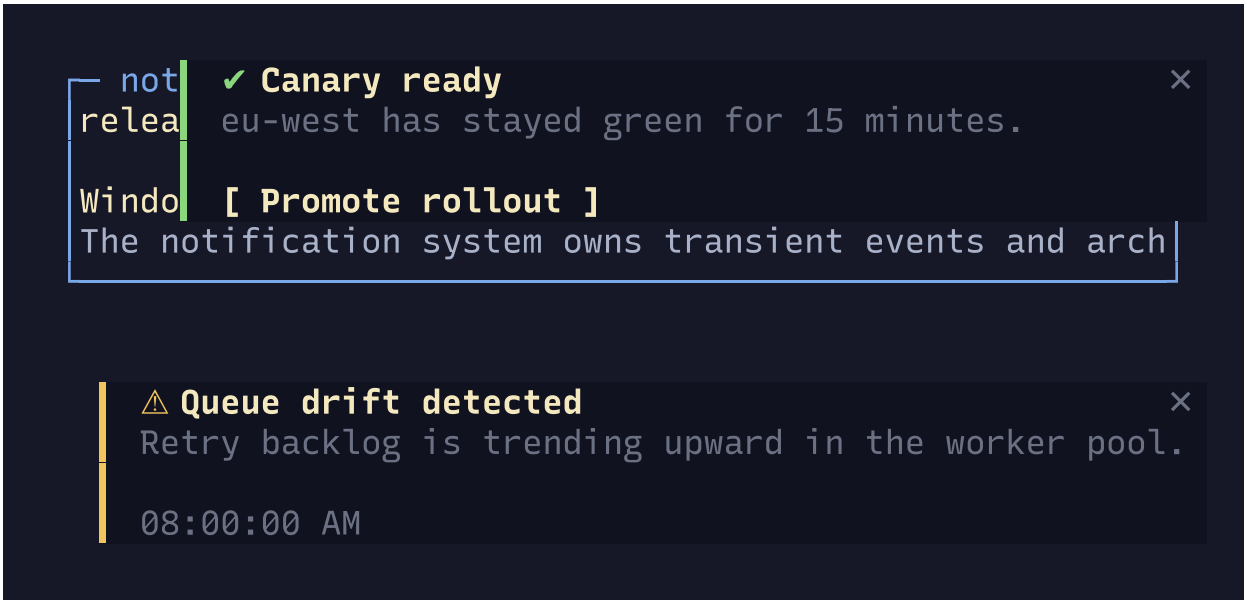
Grouped keyboard reference plus compact shell hint surfaces for keyboard-owned apps that need shortcut discovery without turning every footer into prose.



Story: help-view; variant: shell-hint; 60 × 7 cells.

renderNotificationStack() / renderNotificationHistorySurface() / createFramedApp()

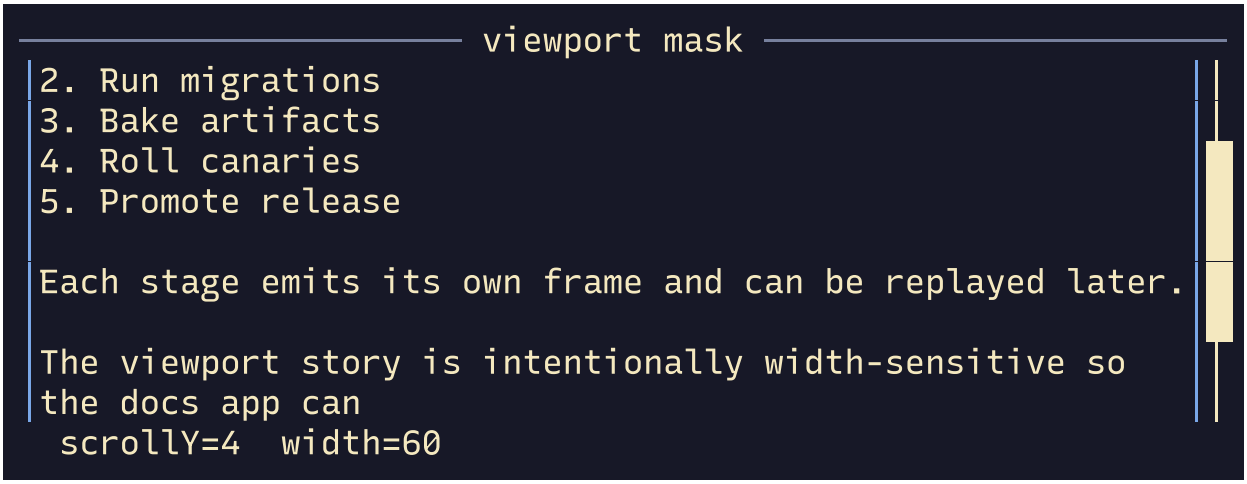
Shell-owned transient messaging system with stacked live notifications, explicit placement, actions, archived review history, and framed runtime routing.



Story: notification-system; variant: live-stack; 60 × 14 cells.

viewportSurface() / pagerSurface() / focusAreaSurface()

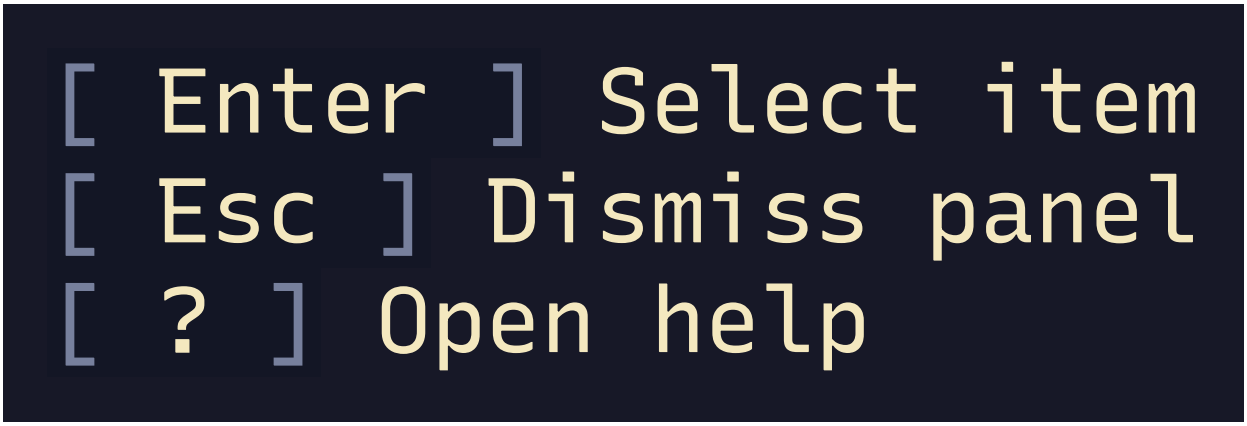
Masking and scrollable-pane family for bounded overflow content, long linear readers, and focused inspection panes without flattening structured surfaces first.



Story: viewport-surface; variant: document; 60 × 11 cells.

kbd()

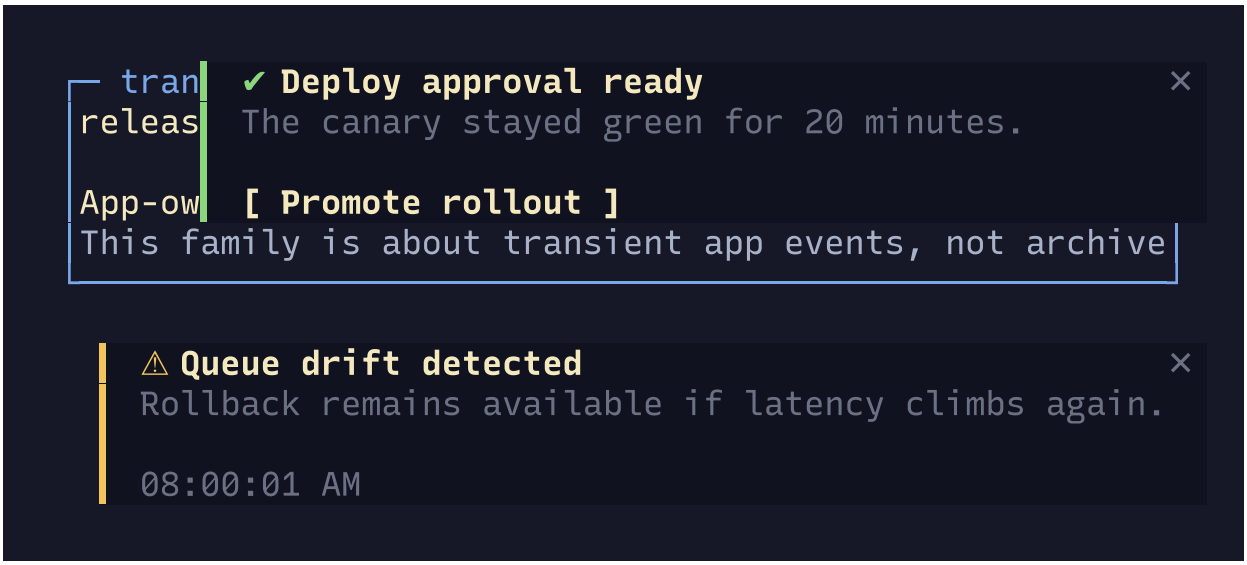
Inline shortcut cue for local actions that should stay adjacent to the thing they affect.



Story: kbd; variant: local-actions; 21 × 3 cells.

pushNotification() / renderNotificationStack()

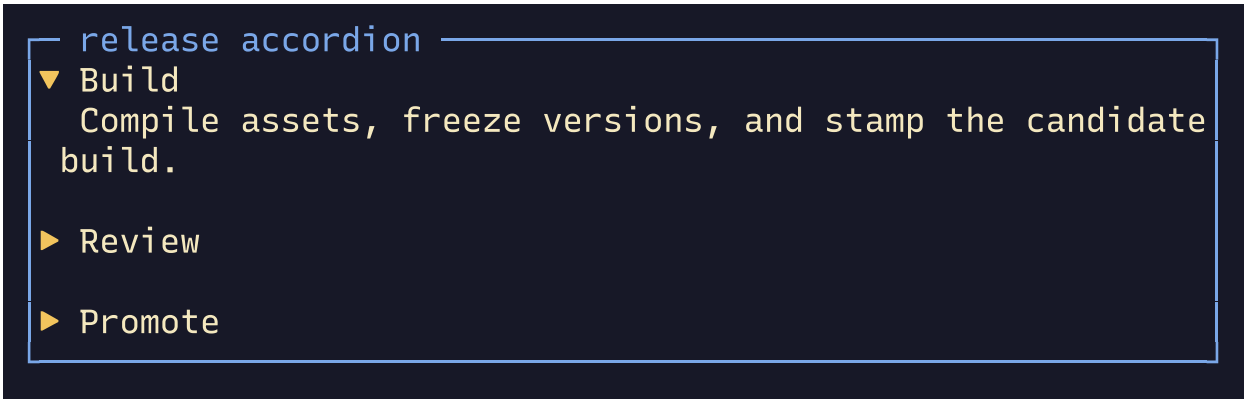
App-owned transient notification family for live stacked events, tones, actions, and placements before archived review or broader shell routing becomes the main concern.



Story: transient-app-notifications; variant: actionable-live; 60 × 13 cells.

accordion() / interactiveAccordion()

Progressive-disclosure family for scanning summaries first and revealing detail only when the user chooses to open a section.



Story: progressive-disclosure; variant: summary-first; 60 × 9 cells.

breadcrumb() / stepper() / paginator()

Wayfinding and progress family for showing where the user is, what stage they are in, or how far through a sequence they have moved.



Story: path-and-progress; variant: wayfinding; 60 × 5 cells.

loadRandomLogo() / gradientText()

Expressive branding family for rare logo and gradient emphasis moments that add atmosphere without becoming routine application chrome.



Story: expressive-branding; variant: launch-moment; 60 × 6 cells.

renderByMode()

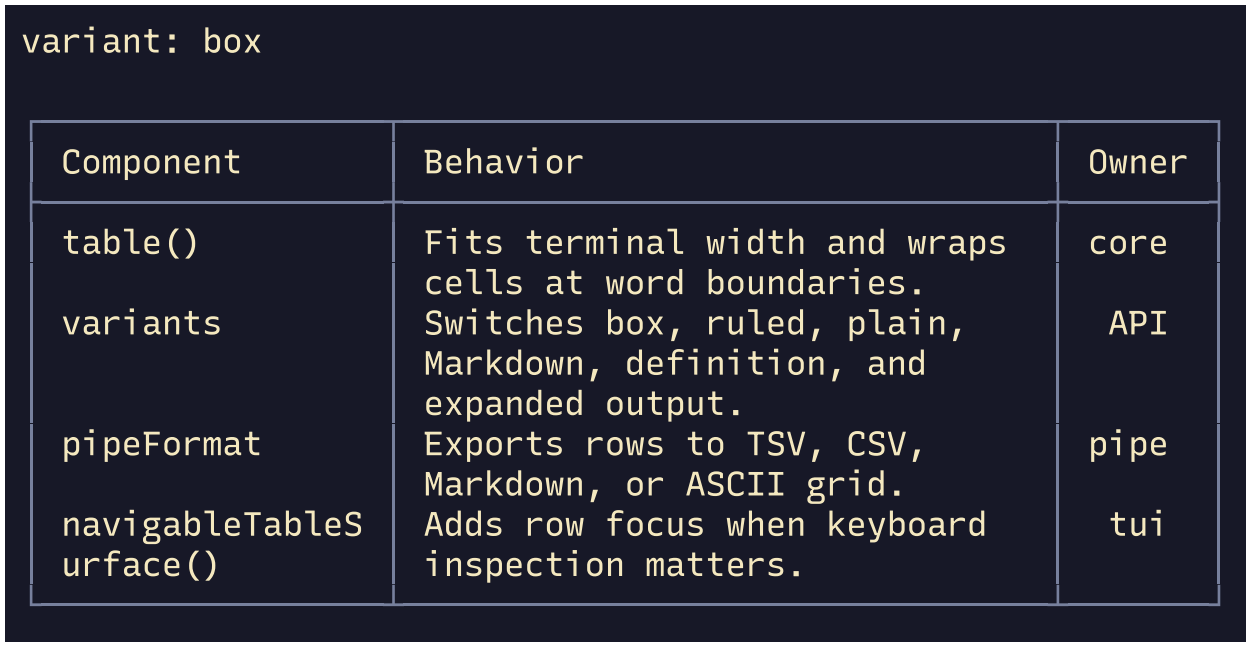
Authoring seam for app-defined primitives that keep one semantic meaning while lowering honestly across interactive, pipe, and accessible modes.



Story: mode-aware-custom-primitives; variant: deployment-spark; 56 × 4 cells.

table() / navigableTableSurface()

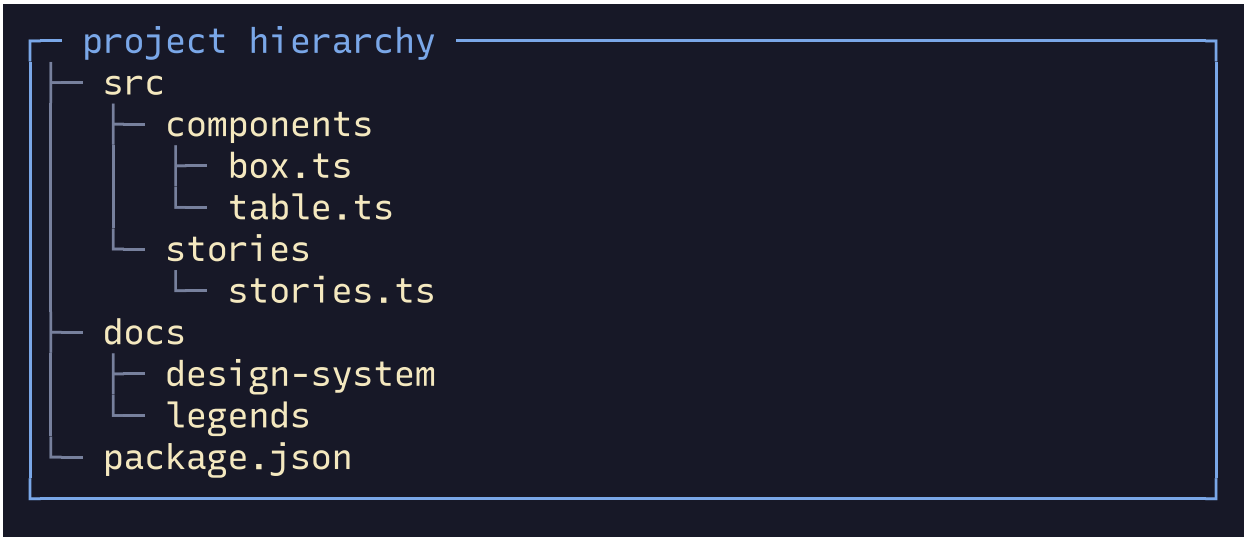
Dense comparison family for row-and-column inspection, responsive table variants, explicit pipe serializations, and keyboard-owned row focus.



Story: dense-comparison; variant: box; 60 × 15 cells.

tree() / filePickerSurface()

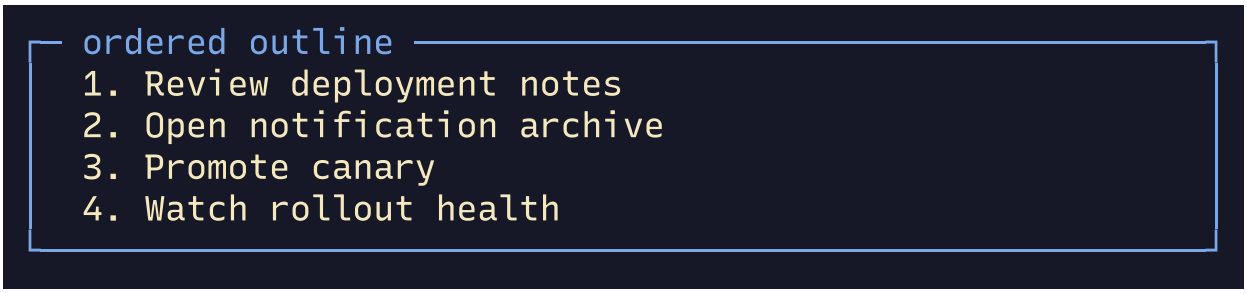
Hierarchy family for parent-child structure and filesystem-style browsing where nesting, path context, and directory boundaries matter more than tabular comparison.



Story: hierarchy; variant: project-tree; 58 × 12 cells.

enumeratedList() / browsableListSurface()

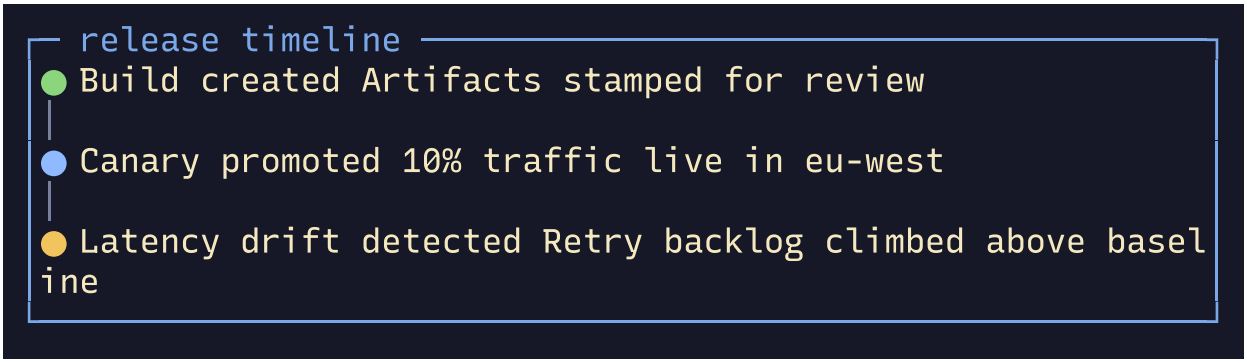
Exploration-list family for one-dimensional scanning where item order and focused inspection matter more than columns or hierarchy.



Story: lists-for-exploration; variant: ordered-outline; 58 × 6 cells.

timeline() / dag()

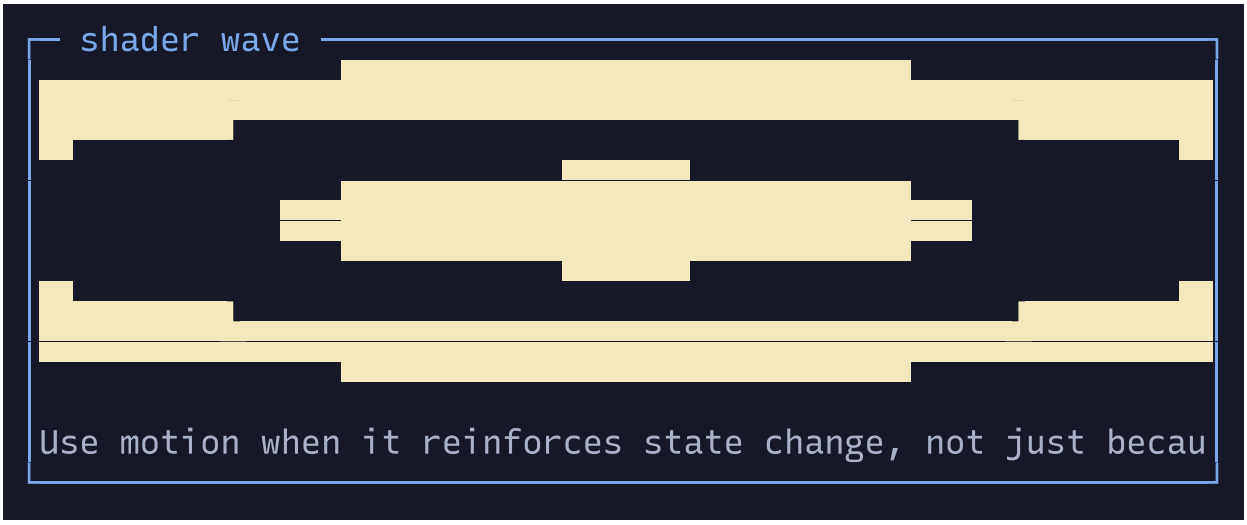
Temporal and dependency family for explaining what happened next or what depends on what when order or causality is the real structure.



Story: temporal-or-dependency-views; variant: release-timeline; 60 × 8 cells.

canvas()

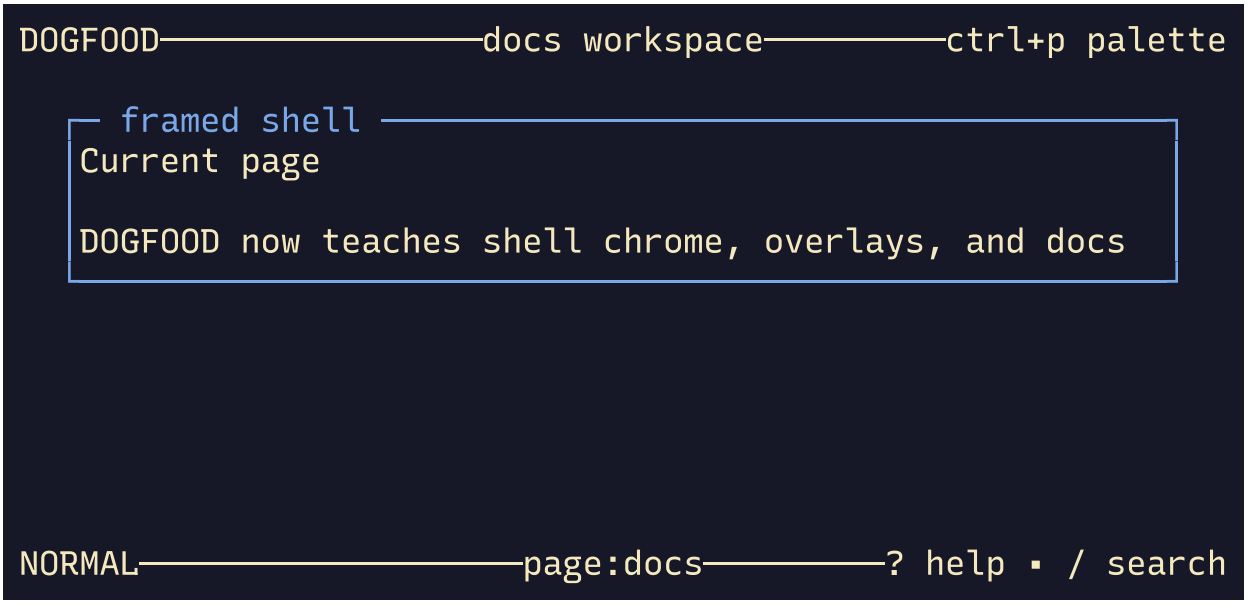
Motion and shader family for deliberate visual emphasis, transitions, and animated atmosphere when the effect reinforces state change or product voice instead of competing with the task.



Story: motion-and-shader-effects; variant: shader-wave; 60 × 12 cells.

createFramedApp() / statusBarSurface() / commandPaletteSurface()

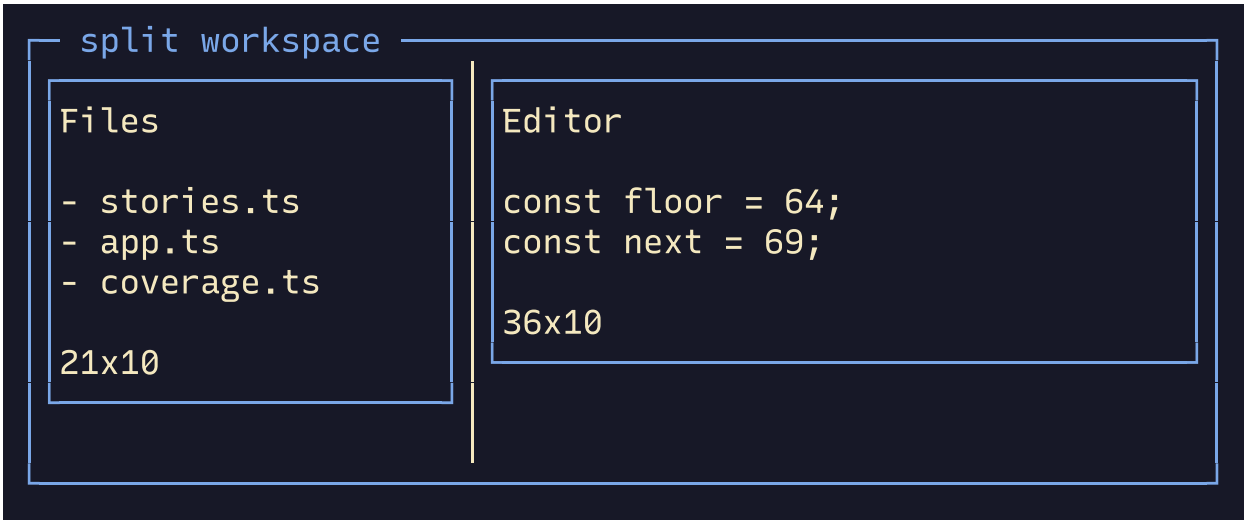
App-shell family for multi-view framed applications with global status, command discovery, overlays, and consistent shell-owned chrome.



Story: app-shell; variant: framed-page; 60 × 14 cells.

splitPaneSurface() / gridSurface()

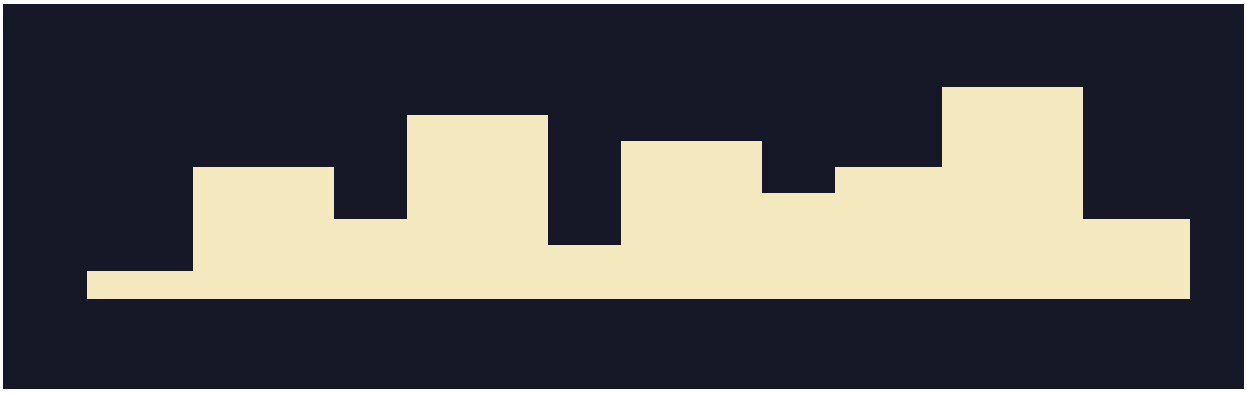
Workspace-layout family for honest spatial composition when simultaneous context materially helps the task and sequential flow would hide important relationships.



Story: workspace-layout; variant: split-context; 60 × 12 cells.

sparkline()

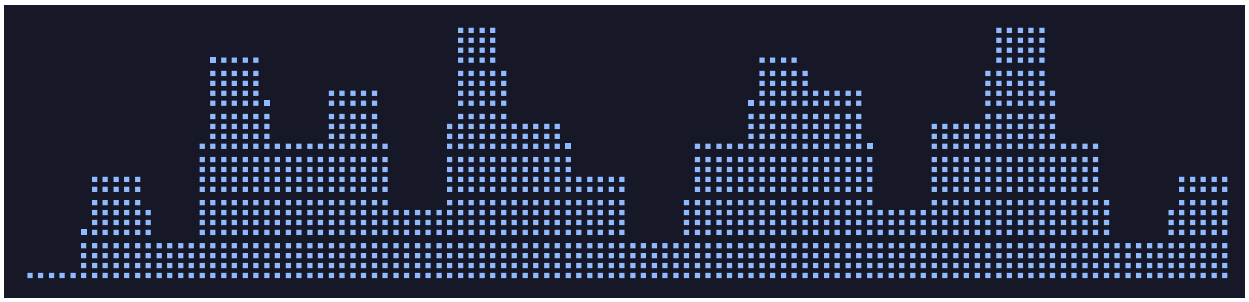
Compact inline trend graph using Unicode block characters — a glanceable shape next to a label instead of a number alone.



Story: sparkline; variant: basic; 10 × 1 cells.

brailleChartSurface()

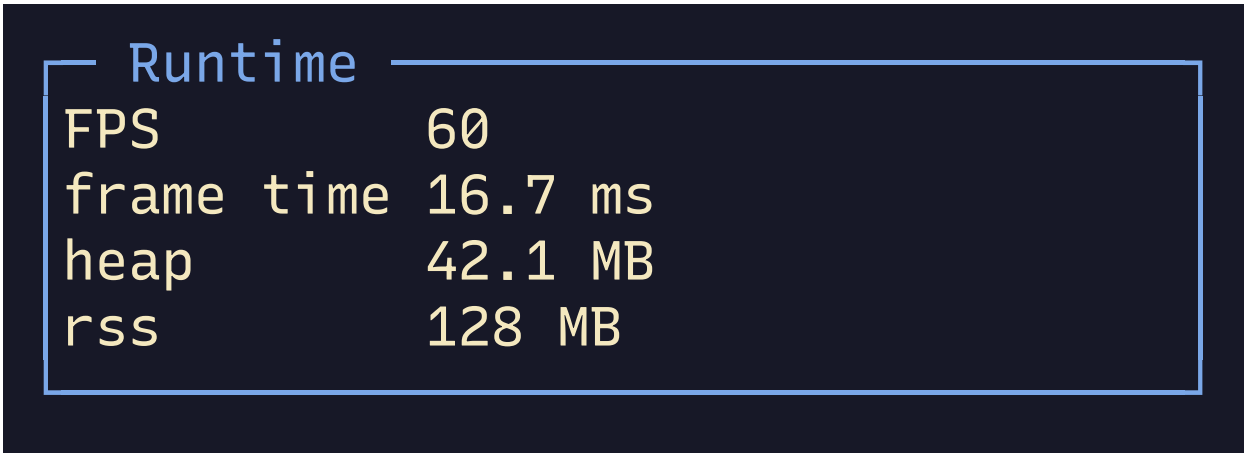
High-density filled area chart using Unicode Braille characters — 2×4 sub-pixel resolution per terminal cell for smooth curves in tight space.



Story: braille-chart; variant: basic; 56 × 6 cells.

statsPanelSurface()

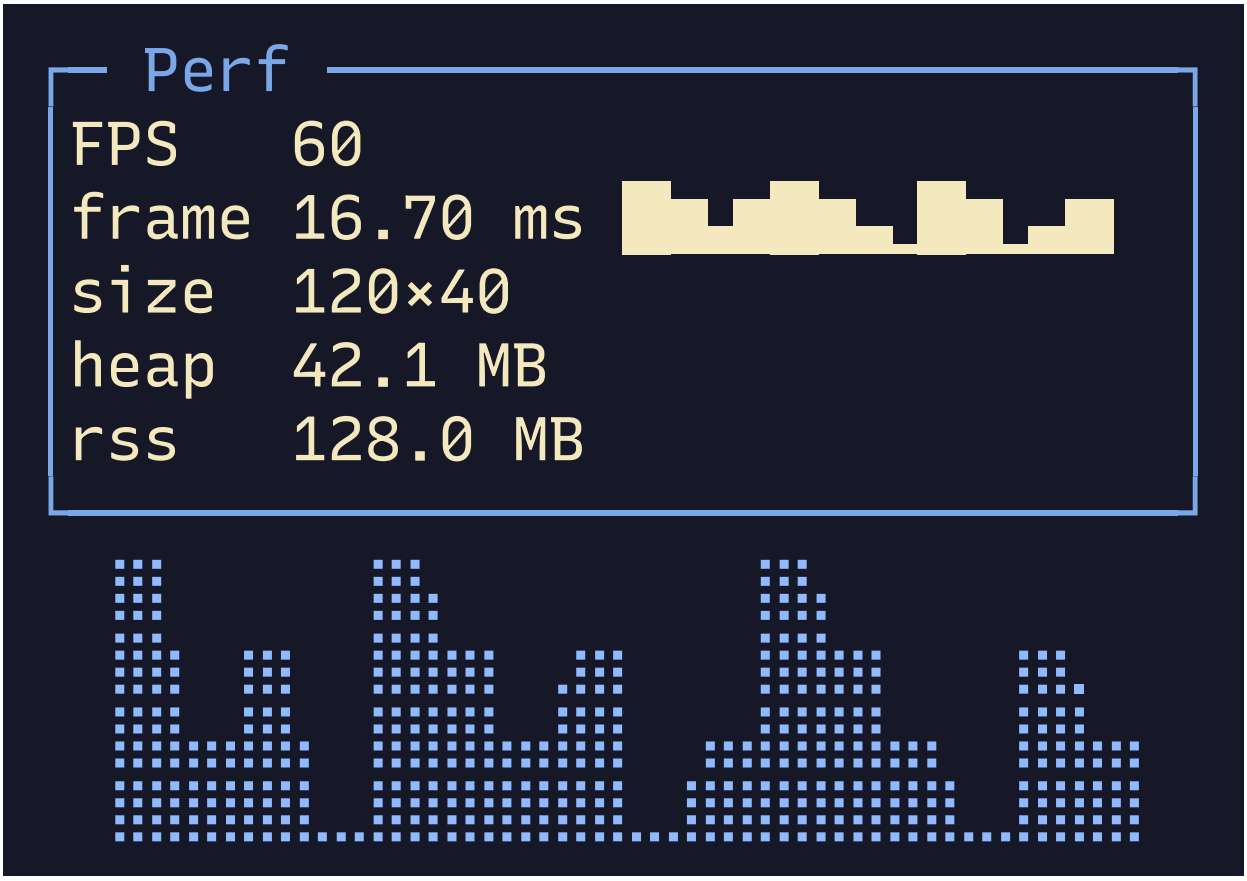
Titled bordered panel with aligned key-value metric rows and optional inline sparklines — the go-to component for labeled metric groups.



Story: stats-panel; variant: basic; 36 × 6 cells.

perfOverlaySurface()

Prebuilt FPS + memory dashboard composing statsPanelSurface and brailleChartSurface — drop-in performance overlay for any app.



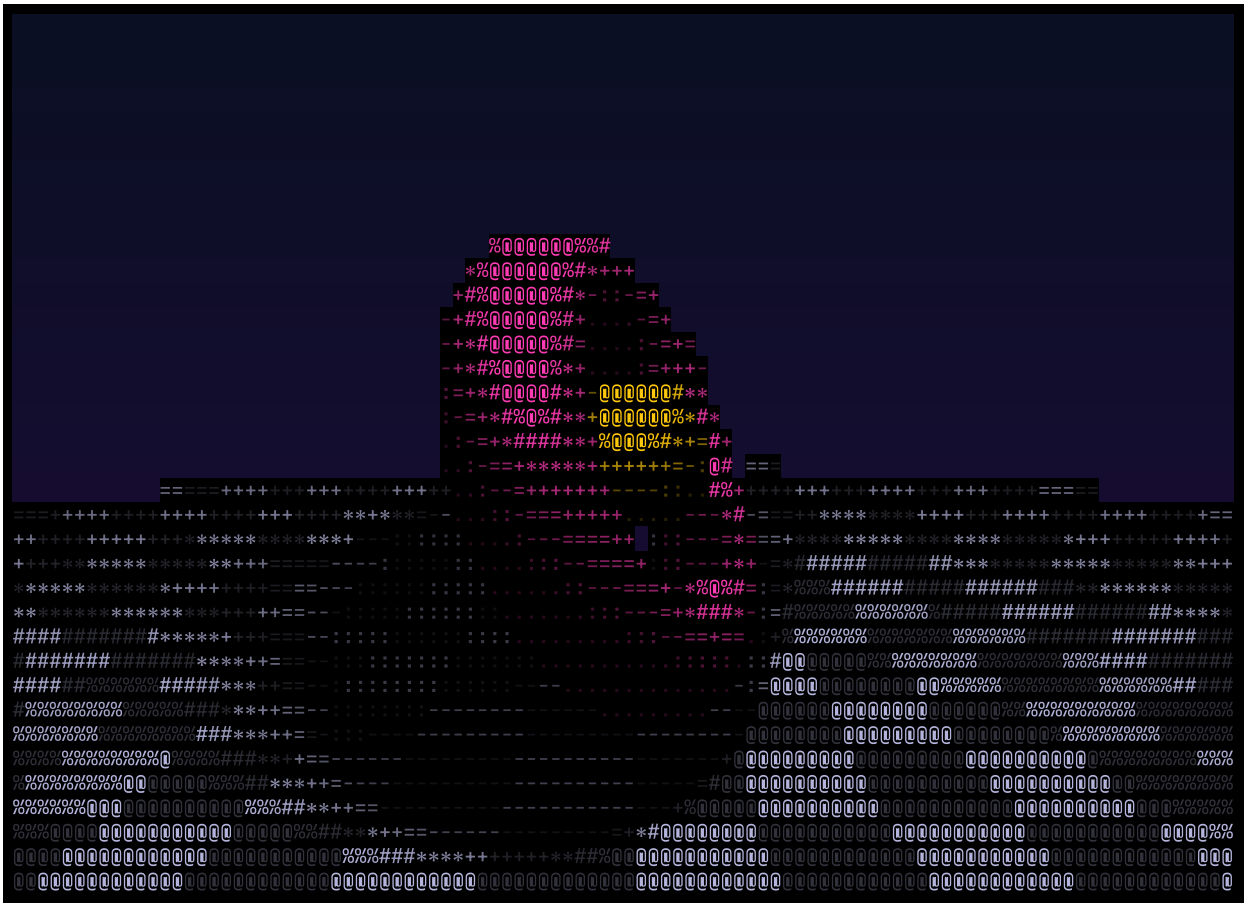
Story: perf-overlay; variant: basic; 32 × 11 cells.

Renderer experiments from bijou-fun

These two captures come from the separate bijou-fun demonstration application. Its registry contains gradients, a wormhole, fluid simulations, raymarching, a raycaster, font rendering, a text cylinder, metaballs and a raytracer. The inspected checkout was 0685ddc993491d309468b4f3ca7bc5e6a6e5f4b0; its installed Bijou package was 7.1.0. The source modules were called directly with a controlled viewport and input values; the network checkout was left unchanged.

Raymarching in terminal cells

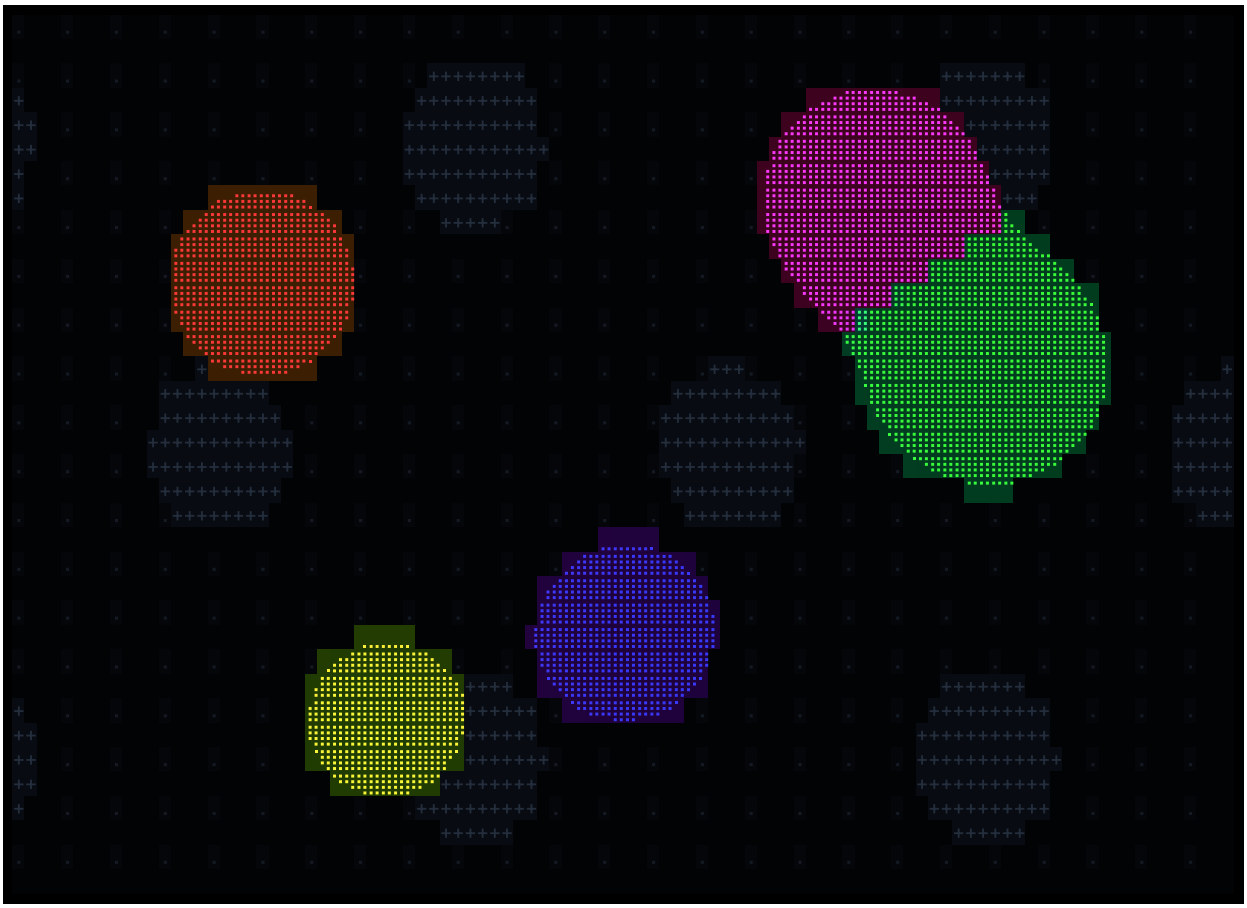
Lighting becomes glyph and color



The raymarching example evaluates a signed-distance scene, estimates normals, combines lighting and shadows, then chooses an ASCII shading character and RGB values per cell. This is an actual Surface capture at 100 by 36 cells, frame 90 and elapsed time 1.5 seconds. The in-app control panel was disabled for the capture. Recursive Mono supplies the printed glyphs; the scene geometry and colors come from the demo.

Metaballs and a denser subgrid

Sampling below a character cell



The metaballs example evaluates a binary subgrid with two samples across and four down for each terminal cell, then turns that pattern into a Braille character. It blends colors from the contributing fields. This capture uses the same viewport, frame and elapsed time, with the source’s fixed default blob positions and the control panel hidden. DejaVu Sans supplies Braille symbols missing from Recursive.

What an illustration establishes

A capture has a scope

These figures show two deterministic rendered states. They do not establish frame rate, sustained resource use, interaction quality or cross-version compatibility. The source hashes, parameters and cell hashes are retained in demo-manifest.json. They are useful examples of how a renderer can make an image from terminal cells, alongside the more task-oriented job-monitor workshops.

Glossary

Terms in this glossary describe the teaching companion and the vocabulary used to explain it. An identifier in a historical record can have a different version-specific meaning.

Adapter

Code that connects a host or framework boundary to application contracts. The companion has keyboard, scheduler and shutdown adaptations.

App

The Bijou runtime contract implemented by init, update and view in the companion. AppModel includes viewport state around the task model.

Application model

A snapshot of facts used to decide behavior and presentation. The companion separates the Monitor task model from viewport dimensions and scroll offset.

Cancellation

A request to stop owned work. It manages resources; it does not by itself establish that a queued result can no longer arrive.

Cell

A unit in a terminal Surface. A code point, grapheme cluster and occupied terminal width are different concepts.

Checkpoint

A reproducible companion command that exposes a selected behavior of the finished implementation. It is not a separate historical code release.

Cmd

A framework command used by the App to perform work outside a pure transition or request runtime behavior such as quit.

Effect

An application description of work to request, such as starting a particular run or loading logs for a view epoch.

Epoch

A generation number for the current view request. A completion must match the selected job, screen and epoch before becoming visible detail.

Focus and input ownership

The policy determining which active surface receives an intention. Drawing a dialog does not establish this policy.

Grapheme cluster

A user-perceived text unit that can contain multiple code points. The companion wraps clusters and measures their terminal width.

Historical record

A preserved source document whose status and assertions retain their original context. It is not a current implementation guarantee.

Identity

A stable value used to address a job independently of its current position or display label.

Immutable snapshot

A model value that does not change after construction. Later transitions produce another snapshot.

Intention

An application event expressing a requested action, such as start, move or dismiss, after interpreting framework input.

KeyMsg

A decoded framework keyboard message translated by the companion App into an application intention.

Mode

A presentation and delivery choice. The companion offers interactive, static, pipe and accessible modes with distinct output contracts.

Port

An explicit interface to an external capability. The scheduler port permits deterministic tests and a host timer implementation.

Run identity

A generation identifying one attempt at a job. It prevents a completion from an earlier attempt updating a restarted job.

Semantic styling

Choosing appearance from a content role while keeping application meaning independent of color or other decoration.

Stale result

A result whose original owner or generation no longer matches the current model. The companion rejects stale job and view completions.

Surface

Bijou representation of rendered terminal cells used by the companion view and controlled rendering observations.

TEA

The Elm Architecture vocabulary for the state, update and view loop with explicit work outside the transition. The book uses the pinned App contract to make that vocabulary concrete.

Transition

The result of applying an event to a model: a next model and any effect descriptions.

Viewport

The width and height available for the current view, together with the application decisions governing what is visible.

Build stage

An independent source snapshot implementing a declared step of the application. The five stages share a dependency lock but import neither the finished application nor each other. They are distinct from the checkpoint commands that inspect the completed implementation.

Find a practical answer

Getting started

For framework fit and costs, read Choose Bijou deliberately. For installation, the immutable reference and errata, use Version and reproducibility and Workshop 7. Begin the model at Workshop 1.

Input, focus and cancellation

For App, KeyMsg, Cmd, init, update and view, inspect Find the application boundaries and its complete App listing. For Enter, Escape, confirmation and global quit, follow Workshop 3. For completion during confirmation, read the maintenance case study.

Asynchronous work

For run identity, view epoch, stale results and the effect driver, use Workshop 4. For disposal, scheduled work and evidence, read Workshop 6. The troubleshooting chapter distinguishes cancellation requests from acceptance checks.

Output, geometry and styling

For interactive, static, pipe, accessible, TSV and NO_COLOR, read Workshop 2. For Surface, graphemes, cell width, scrolling and semantic styling, use Turn meaning into cells and Workshop 5. For rendered visual examples, consult the component atlas with its capture-version notice.

Tests, releases and repairs

For state, effects, rendering and human observation, read Workshop 6. For regression evidence and upgrades, use Maintain the promise after release. The

source package and licensing obligations are described in Workshop 7 and Licenses and reproduction notices.

Historical reasoning

The separate Design Archive and the complete edition contain the 426 original records, grouped by concern. Locate a record using its original path in the source inventory or the machine-readable manifest. R identities are shared between editions; physical page numbers are edition-specific. A record supplies an argument and evidence to follow, not automatic confirmation that its design is available in the teaching release.

Build stages, exercise repairs and human review

For independent source snapshots and actual diffs, read Build from a working stage. Attempt the faulty fixtures before opening the answer key. The unfamiliar-reader protocol records tasks for a future participant; it is not a completed study.

Exercise answer key

Try each exercise before reading its answer. Run npm run exercise – NAME in a learning copy. A nonzero exit is the deliberate observable failure. Add –repair to select the production path and see the same success criterion pass. The four fixtures live in exercises/run.ts; the repaired behavior lives in src/ and is also present in the final progressive stage.

Modal: *selection moves behind confirmation*

Hint

Inspect the selected ID before and after Down while confirmation is open.

Diagnosis

The faulty fixture creates a model with confirming: false before applying movement. The painted dialog has lost its input boundary.

Repair

Preserve the confirming model and call its transition directly: dialog.update({ type: 'move', delta: 1 }). The production model rejects movement, filtering, starting and log navigation while the dialog owns those intentions. The App translates Enter to confirm-cancel when appropriate.

Why a plausible fix fails

Painting the modal last changes pixels, not event ownership. Blocking movement only in a widget also leaves other model callers able to violate the same contract. Global quit remains an explicit exception.

Success

The selected ID is unchanged, Escape leaves the job running, and confirmation requests cancellation only while the selected job is still running. Run npm

run exercise – modal –repair and the model tests. A screenshot alone cannot establish these observations.

Stale: old logs replace current detail

Hint

Compare the result's job ID and epoch with the current selection and view.

Diagnosis

The faulty fixture assigns late.text directly into the current model. It accepts detail from the previous selection without checking ownership.

Repair

Send the completion through current.update(late) and accept it only when screen, selected ID and epoch still match. Requesting cancellation of the old load is useful resource management; rejecting an old completion is correctness.

Why a plausible fix fails

A longer delay only makes the race less convenient to reproduce. Cancellation can race an already queued result. Matching only the job ID also fails when the same job is revisited under a newer view request.

Success

The old result leaves detail empty; a matching current result can fill it; leaving logs prevents a completion from reopening the screen. Run the repaired stale exercise and inspect the corresponding deterministic transition test.

Color: failure disappears in monochrome

Hint

Strip ANSI sequences and ask which word still identifies failure.

Diagnosis

A red dot and a job name contain no textual failed state. A person or consumer without that color has to guess the meaning.

Repair

Render the model's textual status and failure explanation. The repaired fixture uses the production static formatter, then checks its stripped output for failed. Optional color can reinforce that existing fact.

Why a plausible fix fails

Brighter red, another palette or a legend elsewhere does not place the status in a copied row or machine stream. A passing failed assertion is still a narrow observation; check the error explanation and every promised mode as well.

Success

Failure remains understandable with styling removed, and TSV retains its status and error fields. Run the repaired color exercise and mode tests.

Timer: work emits after its owner closes

Hint

Identify who owns the schedule and where that owner is disposed.

Diagnosis

The faulty branch advances a callback without disposing its driver. Its producer can still emit a progress event after the owning session is closed.

Repair

Call driver.dispose() before advancing the deterministic schedule. The driver cancels pending work and suppresses emissions after disposal. App quit and the host finalization connect application shutdown to that lifecycle.

Why a plausible fix fails

Hiding a pane or discarding an event in the view can leave its producer running. A global job may intentionally survive closing its log pane, so blindly cancelling all work on every screen change is also wrong. The driver belongs to the application session; log requests have a narrower owner.

Success

Advancing the schedule after disposal emits no new events. Cancellation, stale-result rejection and disposal require separate observations. The

repaired timer fixture checks emissions; a PTY check separately examines terminal cleanup.

Verify that a repair preserves a nearby promise

After the repaired fixtures pass, run npm run verify. For the cancellation-race maintenance exercise, keep the test where completion occurs during confirmation: confirming afterwards must close the dialog without overwriting success or emitting an abort. This prevents a repair from changing Escape or cancellation semantics.

An unfamiliar reader walkthrough

An unfamiliar-reader session is planned. The following protocol is prepared; no human observations have been recorded. The companion includes an empty recording form at walkthrough/results-template.json.

Status: **prepared, not conducted**. No participant results are included. An agent running commands is an automated reproduction check, not an unfamiliar human reader.

Facilitation

Use someone who has not authored or rehearsed the example. Give them the exact book edition and companion commit, a clean checkout and the task prompts below. Let them follow the book before offering a hint. Record the page, command, observed result and any assistance. A successful assisted step is different from an independent completion. Leave the original confusing passage identifiable when recording its proposed repair.

Start with the Node/npm versions recorded by the edition. Record terminal, operating system, viewport and any assistive tool. results-template.json is an empty recording form, not a verification receipt. Keep not_run until a real session occurs; do not populate answers with an agent's predictions.

Session A — Build from a clean checkout

1. Find the edition's source and reproduce installation without facilitator help.
2. Run stage 01. Explain which values identify a job and which describe its label.
3. Read the first diff and run stage 02. Identify the failed job without color and describe what pipe mode preserves. Notice that stage 02's interactive specimen is a printed fixture, not a live session.
4. Run stage 03. Start compile, open cancellation, attempt movement, dismiss the dialog, then cancel it. Explain the state after each action.
5. Read the asynchronous step. Run stage 04, open compile logs and select test. Explain why a late response must not replace current detail. Use the test or exercise if wall-clock timing does not make the race visible.

6. Run stage 05 and resize to 24×8. Find the current status and available actions. Inspect a failure and return to another job. Explain what information scrolling preserves and what is no longer simultaneously visible.
7. Attempt one broken exercise before using its answer. Use the hint, then the diagnosis if necessary; record which level was needed.
8. Find the license and the exact companion commit used by the book. Identify one claim the automated tests do not establish.

Completion criteria: installation is reproducible; every stage's declared outcome can be observed; the reader can distinguish state, effects and presentation; all required corrections are attached to a reproducible task and passage. Record failures and assistance instead of turning the criteria into assumed passes.

Session B — Keyboard-only interactive task

Run the final stage in a supported terminal without using a pointer. Ask the participant to start a job, inspect logs, select another job, filter failures, open and dismiss cancellation, and quit. Include a narrow viewport. Record focus confusion, discoverability, hidden controls, misleading status and final terminal state. The completion is a human task observation, not merely a key-routing test.

Session C — Reading with an assistive tool

Use the participant's actual tool and record its name and version. Run the accessible fixture and the color-free static fixture separately. Ask which job failed, why it failed, what is selected, and which actions the text describes. Observe reading order and ambiguity. Do not substitute the expected answer for what the participant actually understood.

The accessible CLI fixture prints once and exits. It does not implement an interactive screen-reader session. Testing a live screen-reader workflow requires an implemented interaction contract and its own task session. Record that item as not_implemented, not as a failed or passed snapshot-reading observation.

Turn observations into repairs

For every problem retain: edition, source commit, task, page, exact command, expected observation, actual observation, assistance, reproduction and proposed change. Distinguish a code defect, an explanation gap, an environment mismatch and an unimplemented capability. After a repair, repeat the affected task; use another unfamiliar participant before calling its discoverability independently verified.

Sources and further reading

The historical source inventory belongs to the separate Design Archive and the complete edition. It accounts for 426 original paths and their hashes. This teaching volume uses the same frozen corpus and shared illustration assets without printing the historical records.

The companion source is fixed to the commit and release recorded on the version page. Its LICENSE, NOTICE, VERIFICATION.json, checkpoints and exercises belong to that edition. Book source manifests identify the cell captures, tables, diagrams, fonts and archived documents. Use the archive when following the history of a design; use the pinned companion when reproducing a workshop.

P2

Licenses and reproduction notices

Code example license

The following is the full Apache License, Version 2.0, including its application appendix, retrieved from the Apache Software Foundation. A byte-identical copy is distributed as licenses/Apache-2.0.txt and with the companion application's source package.

Apache License
Version 2.0, January 2004
http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

   "License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

   "Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

   "Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

   "You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

   "Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

   "Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

   "Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

   (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

   (b) You must cause any modified files to carry prominent notices stating that You changed the files; and

   (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of

the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity,

or other liability obligations and/or rights consistent with this
License. However, in accepting such obligations, You may act only
on Your own behalf and on Your sole responsibility, not on behalf
of any other Contributor, and only if You agree to indemnify,
defend, and hold each Contributor harmless for any liability
incurred by, or claims asserted against, such Contributor by reason
of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following
boilerplate notice, with the fields enclosed by brackets "[]"
replaced with your own identifying information. (Don't include
the brackets!) The text should be enclosed in the appropriate
comment syntax for the file format. We also recommend that a
file or class name and description of purpose be included on the
same "printed page" as the copyright notice for easier
identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

Attribution and presentation changes

Bijou Book and Design Archive
Copyright 2026 James Ross.

Code examples reproduced or adapted from Bijou retain the Apache License, Version 2.0. Original sources and exact source hashes are recorded in source-manifest.json and the book source inventory. The source-corpus.json archive preserves original source notices and examples without changes.

Book presentation changes (2026): source chapters were regrouped into parts; editorial introductions and headings were added; conceptual arrow notation was illustrated in TikZ; tables were rendered with the Bijou table component; source code and terminal specimens received separate publication fonts. These presentation changes do not alter the archived source corpus.

No tracked NOTICE file was present in the Bijou source checkout at 836878a8. The upstream project is https://github.com/flyingrobots/bijou.

Font software is separately licensed. Jacquard 24, IBM Plex Mono, VT323 and Recursive are distributed under the SIL Open Font License; their original license files accompany the fonts. DejaVu Sans retains the Bitstream Vera and DejaVu notices in fonts/DejaVu-LICENSE.txt. Recursive Mono files are static instances of Recursive with MONO=1, CASL=0; weights 400/700, slant 0/-15. The instances are publication build assets, not a new font family.

Teaching/archive restructuring (2026): a five-part teaching sequence, connecting explanations, practical reference and historical notices were added. All 426 existing typeset source record bodies remain unchanged. Companion app.ts, main.ts and job.ts listings are reproduced without changes from commit 92864e0f2512f59225b2f1f5b89a0678bd639e62 of https://github.com/flyingrobots/bijou-job-monitor. The full companion LICENSE and NOTICE accompany those excerpts in companion-snippets/.

Font credits

Title and heading type is Jacquard 24. Prose is IBM Plex Mono. Source listings use VT323. Terminal cell projections use Recursive Mono with a linear, fixed-width configuration; DejaVu Sans supplies missing technical symbols. The original font license files are retained in the fonts/ directory, including their copyright holders and attribution notices. These software licenses remain separate from the book content license.